

CSI-5340/ELG-5214

INTRODUCTION TO DEEP LEARNING

MLP, Universal Approximation, Backpropagation

Updates

- Homework 1 due on February 13, 2026
- First presentations on February 12, 2026
- Project proposal due on February 24, 2026

Updates

- Group work - **this should not require facilitation**
 - Please **work collaboratively** and **work out any disagreements cordially**.
- If you have problems:
 - Please **continue to communicate and collaborate with your group member(s) on the discussion board**. This way, we have a record of who is contributing what when it comes time to grade.
 - **If a member is non-responsive or non-cooperative, please document this situation**, let me know, and proceed to try to engage your partners(s) and **make progress on your own if necessary**.
 - Working in diverse groups is a great opportunity to learn and an essential life skill. **I will take any failure to participate very seriously.**

Project Guidance

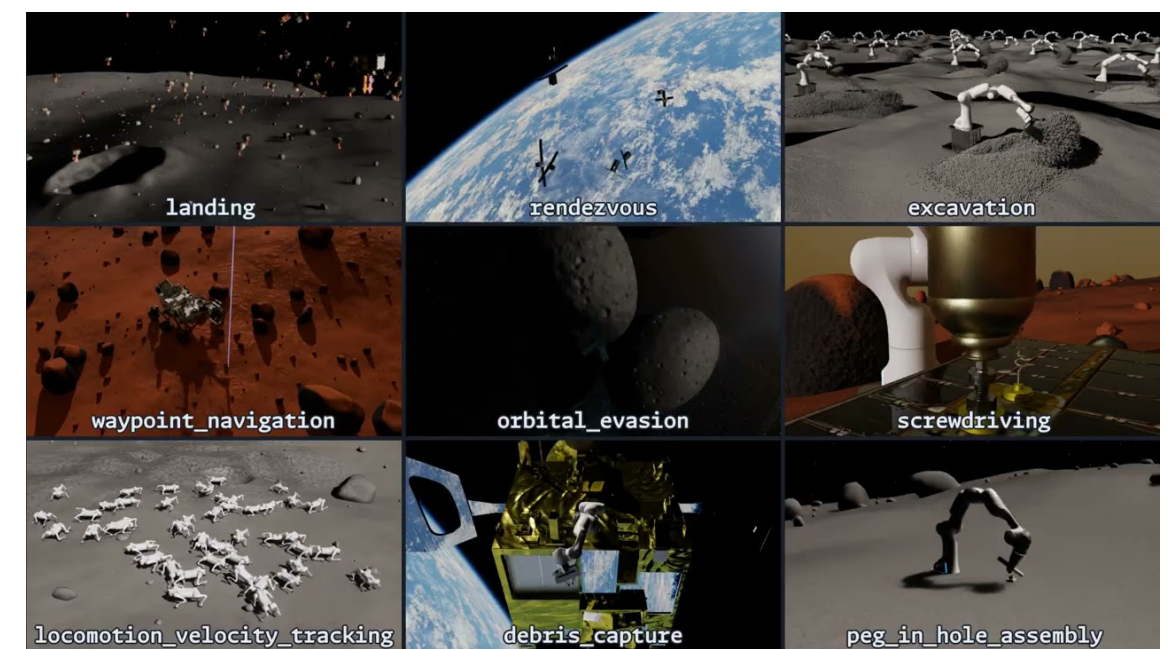
- Key Things to Think About When **Developing a Research Question**
 - **Relevance & Motivation**
 - Why is this question worth answering?
 - Does it address a gap, limitation, or open challenge in the RL literature or an application?
 - **Feasibility**
 - Can the project be completed within ~8-10 weeks with 4 students?
 - Are computational resources sufficient?
 - Is the environment complexity appropriate?
 - **Novelty / Contribution**
 - Is the idea a meaningful variation, extension, or evaluation?
 - Are you proposing: a new method, an improvement, a new environment/task, or a thorough empirical study?
 - **Measurability**
 - What metrics quantify success?
 - What baselines will you compare against?
 - **Clarity**
 - Can you express the question as a clear, testable hypothesis or objective?

Project Guidance

- Key Things to Think About When **Answering a Research Question**
 - **Literature Review**
 - Understand and properly reference prior work: what methods, limitations, and evaluation protocols already exist?
 - **Well-Designed Experiments**
 - Identify baselines and ablations.
 - Ensure reproducibility: fixed seeds, clear pipelines, and versioned code.
 - **Proper Evaluation**
 - Multiple random seeds.
 - Statistically meaningful results.
 - Consider sample efficiency, stability, robustness, generalization.
 - **Diagnostics**
 - Plot learning curves.
 - Track Q-value estimates, entropy, reward components, policy divergence.
 - Identify failure modes.
 - **Interpretation & Insight**
 - Go beyond “it works / doesn’t work”, “A is better than B” —explain why.
 - **Limitations & Future Work**
 - What constraints or assumptions matter for interpreting results?

Project Guidance

- **High-Level RL** Topics Suitable for Graduate Projects
 - Model-based RL (Dreamer, MuZero-style architectures)
 - Example ideas
 - Design a new MBRL method with modern DL
 - Design a new exploration strategy for MBRL (for example incorporate intrinsic motivation)
 - Compare SOTA MBRL methods on a novel task...
space GYM
 - Analyze inter-seed stability of MFRL and MBRL, and understanding the differences



Project Guidance

- **High-Level RL** Topics Suitable for Graduate Projects
 - Model-based RL (Dreamer, MuZero-style architectures)
 - Offline / Offline-to-online RL
 - Hierarchical RL (options, skill discovery)
 - Meta-RL (learn-to-learn)
 - Exploration strategies (curiosity, novelty search, RND)
 - Safe RL (constraints, risk sensitivity, Lyapunov methods)
 - RL for continual learning
- **Advanced Topics**
 - RL with large language models
 - Distributional RL
 - Representation learning for RL (contrastive, world models)
 - RL in partially observable settings (POMDPs, state estimation)
 - Generalization, overfitting, and robustness

Project Guidance

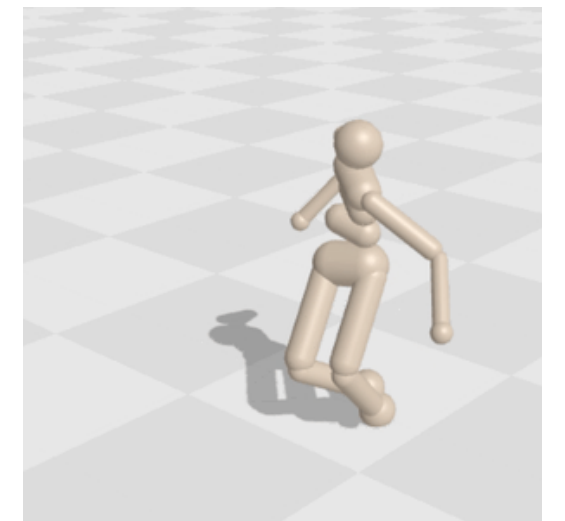
- **Exploring Modern DL Methods in Deep RL** Research Questions
 - **Transformers or Diffusion for Exploration**
 - Can generative models learn priors over “interesting” states and guide agents toward novel or informative regions of the environment?
 - **Transformers for Skill Discovery or Hierarchical RL**
 - Can transformers discover reusable skills by attending to temporal patterns in trajectories?
 - Can diffusion models generate primitive behaviours that form the building blocks of hierarchical policies?
 - **Modern DL for Safe RL**
 - Can generative models predict unsafe states ahead of time and integrate those predictions into constrained policy optimization?
 - Can transformers learn risk-aware representations for more reliable safety constraint satisfaction?
 - **Continual & Lifelong RL with Transformers**
 - Do transformer architectures mitigate catastrophic forgetting by separating context-specific patterns across tasks?
 - Can attention-based replay or generative replay improve continual learning performance?
 - **Large Models as World Models or Reward Models**
 - Can pretrained vision-language or multimodal transformers serve as reward models, world models, or representation encoders for RL agents?
 - How do foundation models influence sample efficiency or generalization?

Project Guidance

- **Exploring Modern DL Methods in Deep RL** Research Questions
 - **Transformers for Policy or Value Function Architectures**
 - Can sequence-modeling architectures (e.g., Decision Transformer, GTrXL) improve long-horizon credit assignment or stability compared to standard MLP/CNN policies?
 - Do transformer-based world models yield better planning or representation learning than recurrent models (LSTMs, GRUs)?
 - **Transformers for Offline RL**
 - Can transformer-style sequence modeling outperform temporal-difference learning on large offline datasets with suboptimal trajectories?
 - How does attention help extrapolate to unseen state-action sequences?
 - **Diffusion Models for Generating Trajectories or Policies**
 - Can diffusion models generate realistic action trajectories to support planning, imitation learning, or offline RL?
 - Can denoising diffusion policies improve exploration by sampling diverse, high-quality action sequences?
 - **Diffusion Models for Environment or Dynamics Modeling**
 - Can diffusion-based dynamics models yield more accurate stochastic predictions than Gaussian world models, improving model-based RL performance?

Project Guidance

- Possible Pre-built Environments (JAX)
 - Gymnax
 - JAX-optimized classic control, BSuite behaviors.
 - Good for reproducible experiments and large-scale sweeps.
 - Brax
 - Differentiable physics engine (fast, parallelizable).
 - Popular for continuous control and model-based RL.
 - Playground
 - Robot learning framework built with MJX
 - Goal of streamlining simulation, training, and sim-to-real transfer onto robots.
 - PufferLib
 - JAX-friendly wrappers for large RL environments (Atari, Minigrid, DMControl).



Project Guidance

- Widely Used RL Environments (non-JAX)
 - Classic & General
 - OpenAI Gym / Gymnasium (classic control, Box2D, Atari)
 - Atari ALE
 - Minigrid (symbolic reasoning tasks)
 - Continuous Control
 - DMControl / DeepMind Control Suite
 - Mujoco (robotics & physics-based control)
 - PyBullet (open-source robotics)
 - More Complex Tasks
 - Procgen (procedural generalization tasks)
 - MineRL (Minecraft)
 - CARLA (autonomous driving)
 - Habitat / IsaacGym (embodied AI, robotics)

Project Guidance

- Guidelines for **Developing Your Own RL Environment**
 - **Start Simple:** Minimal state and action spaces. Incrementally add complexity.
 - **Clearly Define the RL Components**
 - State space: What information does the agent observe?
 - Action space: Discrete/continuous?
 - Reward function: Dense vs. sparse; avoid reward hacking.
 - Episode termination conditions.
 - **Align Difficulty with Research Goals**
 - Don't make the environment so hard that you can't focus on the algorithm.
 - Ensure a reasonable learning signal.
 - **Determinism vs. Stochasticity**
 - Deterministic environments simplify debugging.
 - Add stochasticity for robustness/generalization studies.
 - **Provide Logging and Visualization**
 - Expose internal state for debugging.
 - Allow rendering or trajectory replay.
 - **Ensure Computational Efficiency**
 - Should run thousands of episodes in reasonable time.
 - Avoid slow Python loops; consider vectorization if possible.
 - **Document the API:** Match Gymnasium-style structure
 - **Include Baseline Agents:** Always include a simple heuristic or random agent to test the environment.

Outline

- Warm Up: Logistic/Soft-Max Regression
- MLP
- Universal Approximation Theorem
- Backpropagation

Multi-Layer Perceptron

Logistic/Soft-Max Regression

Binary Classification

- Start with the **binary classification** case
- Let $D := \{(x_i, y_i) : i = 1, 2, \dots, N\}$ be set of tuples, where
 - Each x_i is specified as a vector in $\mathbb{R}^m$
 - Each y_i is the class label
- **The objective:** Develop a classifier which assign random examples X (possibly unseen) to a class label Y .

$$D_X = \left\{ \text{Cat}, \text{Dog} \right\}$$

$$D_Y = \left\{ \begin{array}{cc} \text{Cat} & \text{Dog} \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ \dots & \dots \\ 0 & 1 \\ 0 & 1 \end{array} \right\}$$

$$\hat{y} = f(x)$$

Binary Classification

- *Discriminative modelling strategy*

- **(Modelling)** Choose a family $\mathcal{H}$ of hypotheses for conditional distribution $p_{Y|X}$
- **(Optimization)** Find the “best” distribution $\hat{p}_{Y|X}$ in $\mathcal{H}$ based on D

$$D_X = \left\{ \text{cat, cat, ..., dog, dog} \right\}$$

$$D_Y = \{\text{cat, cat, ..., dog, dog}\}$$

“best” in what sense?

Binary Classification

- Solving with a *discriminative modelling strategy*
 - **(Modelling)** Choose a family $\mathcal{H}$ of hypotheses for conditional distribution $p_{Y|X}$
 - **(Optimization)** Find the “best” distribution $\hat{p}_{Y|X}$ in $\mathcal{H}$ based on D

$$X = \left\{ \text{cat, cat, ..., dog, dog} \right\}$$
$$Y = \{\text{cat, cat,, dog, dog}\}$$

“best” in what sense?

- **Usual choice:** Maximum Likelihood Estimator (MLE)
 - Maximize the likelihood of observed data.
 - Sensitive to noise and overfitting
- **More robust choice:** Maximum a-Posteriori Estimator (MAP)
 - Combines likelihood $\frac{p(D | \theta)p(\theta)}{p(D)}$ with prior $p(\theta)$. The prior provides a regularizing factor.

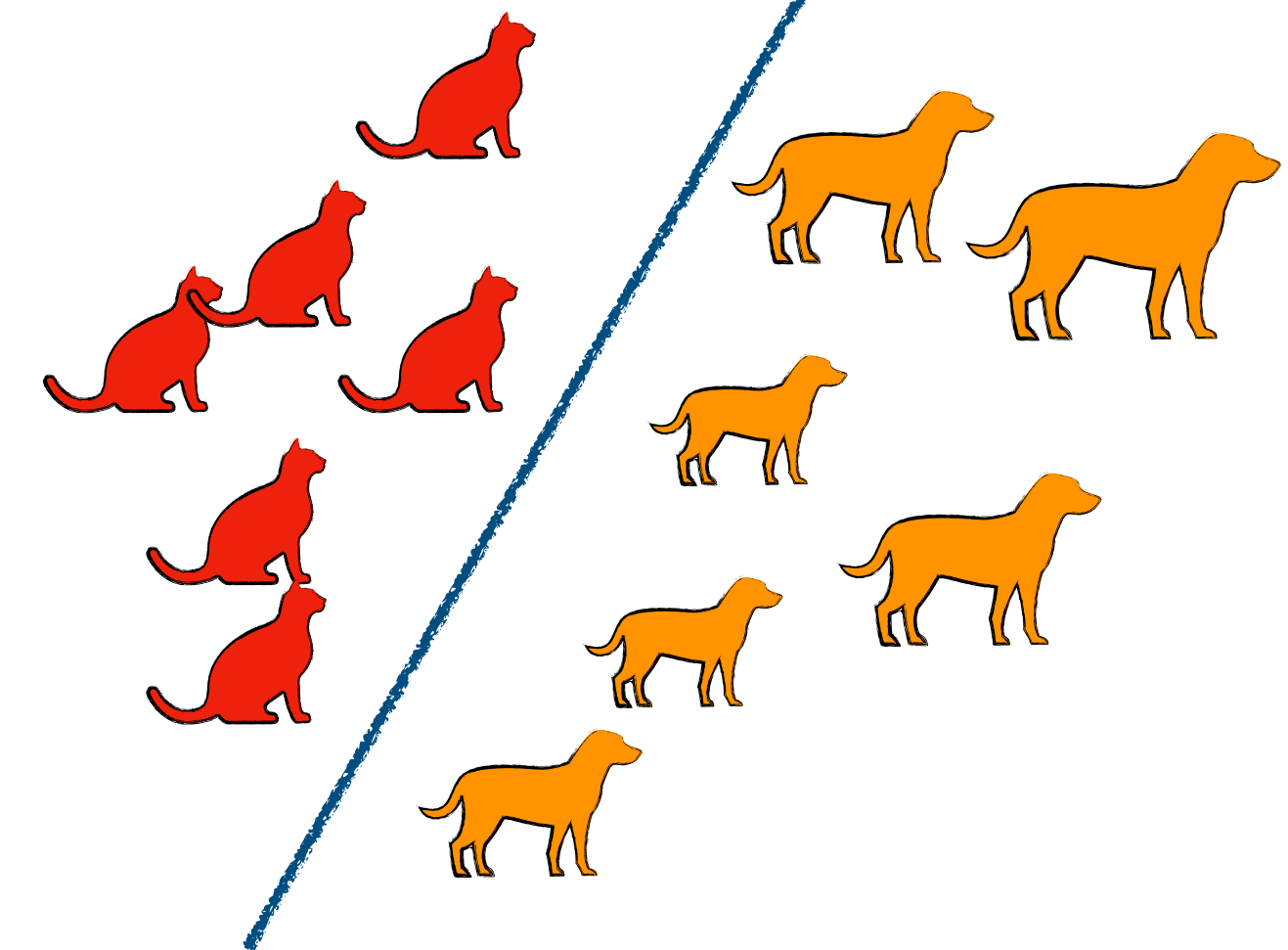
Binary Classification

- Solving with a *discriminative modelling strategy*
 - **(Modelling)** Choose a family $\mathcal{H}$ of hypotheses for conditional distribution $p_{Y|X}$
 - **(Optimization)** Find the “best” distribution $\hat{p}_{Y|X}$ in $\mathcal{H}$ based on D
 - **(Prediction)** For any example X :
 - Declare $\hat{Y} = 1$, if $\hat{p}_{Y|X}(1 | X) \geq 0.5$
 - Declare $\hat{Y} = 0$, otherwise

$$X = \left\{ \text{cat, cat, ..., dog, dog} \right\}$$

$$Y = \{\text{cat, cat, ..., dog, dog}\}$$

$$f(x) = \begin{cases} \text{dog if } \sigma(\theta x + b) \geq \tau \\ \text{cat otherwise} \end{cases}$$



Maximizing likelihood = Minimizing the Cross Entropy Loss

- Logistic regression models $p(Y | X)$ directly
 - Here, MLE objective is equivalent to minimizing cross-entropy (CE) loss
- Why does this matter?
 - Foundation for classification in DL.
 - LG can be seen as a single-layer ANN with a sigmoid or softmax activation.
 - Deep learning extends this idea by stacking multiple layers (MLPs)
 - CE is the standard loss for classification in DL.

Maximizing likelihood = Minimizing the Cross Entropy Loss

- Proof: **Maximizing likelihood** for logistic/softmax regression **is equivalent to minimizing cross-entropy loss**.
- About the proof
 - **ML loss**: Chooses **parameters** that **maximize the likelihood** of observed **data**.
 - **CE loss**: Is derived from the **negative log-likelihood** (NLL)
 - **Minimizing the NLL** is equivalent to **maximizing the log-likelihood**
 - Maximizing $P(y | x)$ (likelihood) is equivalent to minimizing $-\log P(y | x)$

Maximizing likelihood = Minimizing the Cross Entropy Loss

$$\hat{P}_{Y|X}^{ML} = \arg \max_{p_{Y|X} \in \mathcal{H}} p(D | p_{Y|X})$$

$$= \arg \max_{p_{Y|X} \in \mathcal{H}} \prod_{i=1}^N p_{Y|X}(y_i, x_i)$$

$$= \arg \max_{p_{Y|X} \in \mathcal{H}} \log \prod_{i=1}^N p_{Y|X}(y_i, x_i)$$

$$= \arg \min_{p_{Y|X} \in \mathcal{H}} \sum_{i=1}^N \{-\log p_{Y|X}(y_i, x_i)\}$$

$$= \arg \min_{p_{Y|X} \in \mathcal{H}} \sum_{i=1}^N \{-\mathbb{I}\{y_i = 1\} \log p_{Y|X}(1, x_i) - \mathbb{I}\{y_i = 0\} \log p_{Y|X}(0, x_i)\} \quad - \text{Express as Cross-Entropy for binary classification}$$

$$= \arg \min_{p_{Y|X} \in \mathcal{H}} \sum_{i=1}^N \text{CrossEntropy}(\mathbb{I}\{y_i = \cdot\}; p_{Y|X}(\cdot, x_i))$$

$$\bullet \text{ CrossEntropy}(p; q) := - \sum_{y \in \mathcal{Y}} p(y) \log q(y)$$

- The **cross-entropy** between the **true distribution** (given by labels) and the **predicted distribution**
 - p: target distribution; q: predictive distribution.

- Start with MLE

- Assumes (x, y) are sampled i.i.d.

- Take the log (since log is monotonic and simplifies products)

$$(\log(AB) = \log(A) + \log(B))$$

- Maximizing log-likelihood is equivalent to minimizing its negative

Binary Classification

- Maximizing likelihood = minimizing the cross entropy loss
- Proof: Show that **maximizing likelihood** for logistic/softmax regression is equivalent to **minimizing cross-entropy loss**.

$$\hat{P}_{Y|X}^{ML} = \arg \max_{p_{Y|X} \in \mathcal{H}} p(\mathcal{D} | p_{Y|X})$$

$$= \arg \max_{p_{Y|X} \in \mathcal{H}} \prod_{i=1}^N p_{Y|X}(y_i, x_i)$$

$$= \arg \max_{p_{Y|X} \in \mathcal{H}} \log \prod_{i=1}^N p_{Y|X}(y_i, x_i)$$

$$= \arg \min_{p_{Y|X} \in \mathcal{H}} \sum_{i=1}^N \{-\log p_{Y|X}(y_i, x_i)\}$$

$$= \arg \min_{p_{Y|X} \in \mathcal{H}} \sum_{i=1}^N \{-\mathbb{I}\{y_i = 1\} \log p_{Y|X}(1, x_i) - \mathbb{I}\{y_i = 0\} \log p_{Y|X}(0, x_i)\}$$

$$= \arg \min_{p_{Y|X} \in \mathcal{H}} \sum_{i=1}^N \text{CrossEntropy}(\mathbb{I}\{y_i = \cdot\}; p_{Y|X}(\cdot, x_i))$$

$$\bullet \text{CrossEntropy}(p; q) := - \sum_{y \in \mathcal{Y}} p(y) \log q(y)$$

- p: target distribution; q: predictive distribution.

This proof explains why cross-entropy loss is the standard objective for classification in neural networks.

Backpropagation computes gradients of this loss efficiently across layers.

Logistic Regression Model

- **The Logistic function**, a.k.a, sigmoid function:

- Define the sigmoid function:

- $\sigma(x) = \frac{1}{1 + e^{-x}}$

- **Nice derivative property:** $\sigma'(x) := \sigma(x)(1 - \sigma(x))$

- Differentiate:

- $\frac{d}{dx}\sigma(x) = \frac{d}{dx} (1 + e^{-x})^{-1}$

- Apply reciprocal rule:

- $\frac{d}{dx}(u^{-1}) = -u^{-2} \cdot \frac{du}{dx}$, where $u = 1 + e^{-x}$

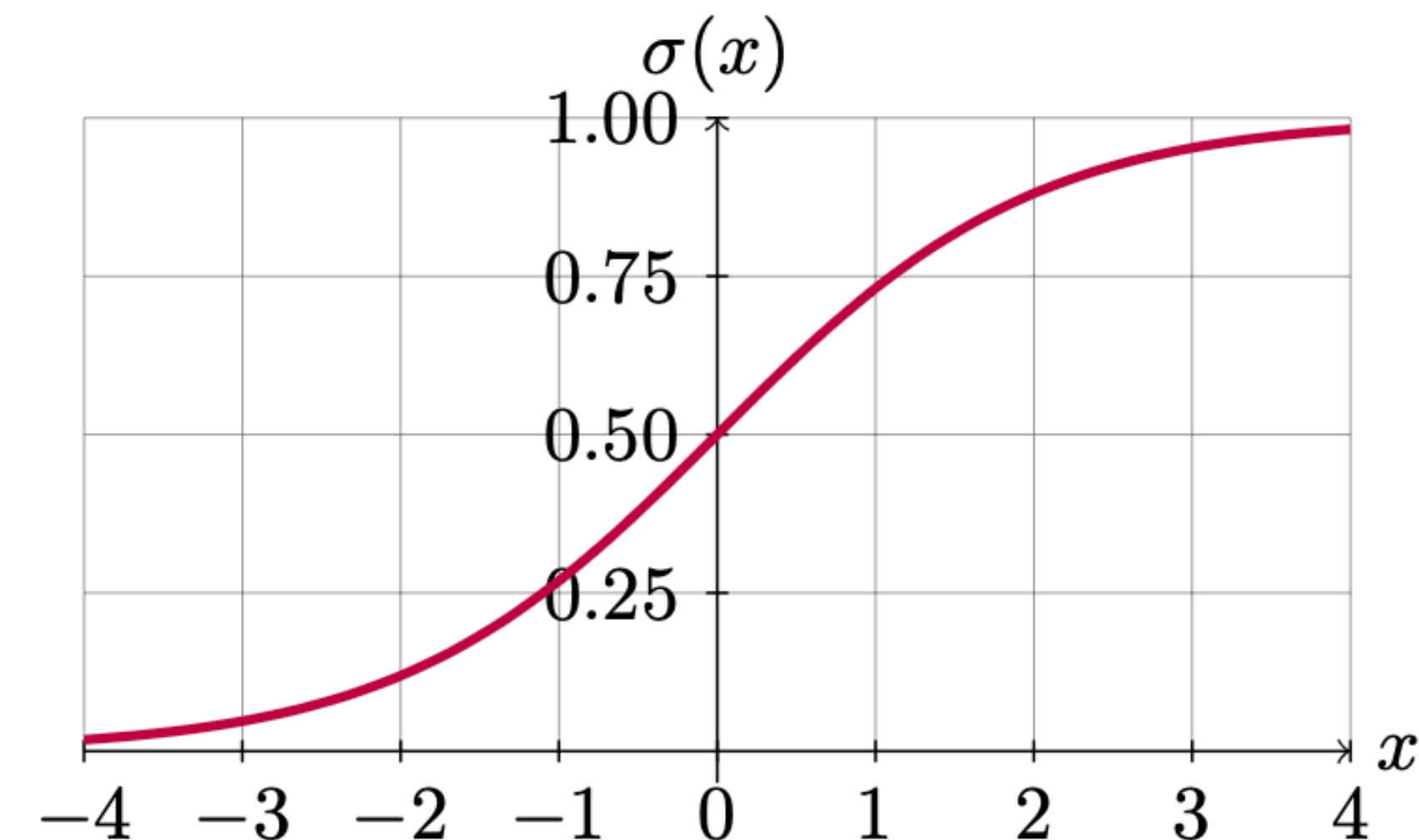
- Compute derivative:

- $\frac{d}{dx}\sigma(x) = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2}$

- Express in terms of $\sigma(x)$:

- $\sigma(x) = \frac{1}{1 + e^{-x}}, \quad 1 - \sigma(x) = \frac{e^{-x}}{1 + e^{-x}}. \quad // \quad 1 - \sigma(x) = \frac{-1 \cdot (1 + e^{-x}) + 1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}}$

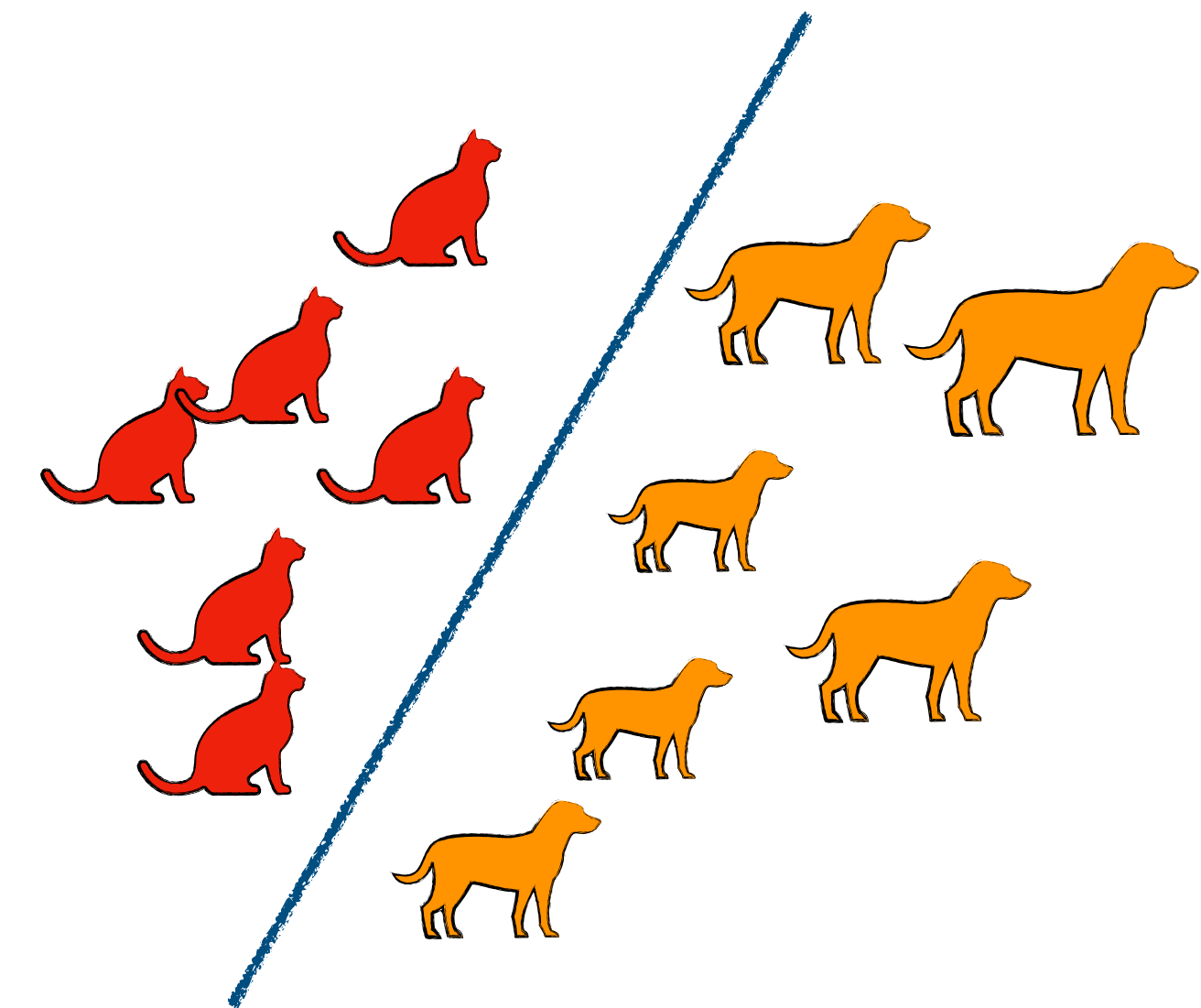
- Final result: $\frac{d}{dx}\sigma(x) = \sigma(x) \cdot (1 - \sigma(x))$



Logistic Regression Model

- Let $\hat{p}_{Y|X} \in \mathcal{H}$ be parametrized by
 - $\hat{p}_{Y|X}(1 | x) := \sigma(w^\top x + b)$
 - $\hat{p}_{Y|X}(0 | x) := 1 - \sigma(w^\top x + b)$
 - where $w \in \mathbb{R}^M$ and b is a scalar. This is known as **the logistic regression model**.
- For notational simplicity, we **may absorb b into vector w and append 1 to vector x** , and write $w^\top x + b$ as $w^\top x$

Geometric Interpretation



$P := \{x \in \mathbb{R}^m : w^\top x + b = 0\}$
marks a hyperplane in $\mathbb{R}^m$ that
cuts the data space $\mathbb{R}^m$ in half

Logistic Regression Model

- **Updated the weights:** according to as the **average cross-entropy loss** over all samples

$$\begin{aligned} L(w) &:= \frac{1}{N} \sum_{i=1}^N \text{CrossEntropy}(\mathbb{I}(y_i = \cdot); p_{Y|X}(\cdot | x_i)) \\ &= -\frac{1}{N} \sum_{i=1}^N \{y_i \log \sigma(w^\top x_i) + (1 - y_i) (\log (1 - \sigma(w^\top x_i)))\} \end{aligned}$$

- We want $w := \arg \min_w L(w)$
- Gradient of the Loss
 - $\frac{dL}{dw} = -\frac{1}{N} \sum_{i=1}^N x_i (y_i - \sigma(w^\top x_i))$
- Gradient Descent Update
 - $w_{\text{new}} := w_{\text{old}} + \lambda \cdot \frac{1}{N} \sum_{i=1}^N x_i (y_i - \sigma(w^\top x_i))$
- Mini-Batch SGD Update
 - $w_{\text{new}} := w_{\text{old}} + \lambda \cdot \frac{1}{B} \sum_{i \in B} x_i (y_i - \sigma(w^\top x_i))$

Multi-Class Classification

- **Generalize to multi-class:** The set $\mathcal{Y}$ of labels contains K elements and $K > 2$.

$$X = \left\{ \begin{array}{c} \text{cat, cat, ..., dog, dog, ..., horse} \end{array} \right\}$$

$$Y = \{\text{cat, cat, ..., dog, dog, ..., horse}\}$$

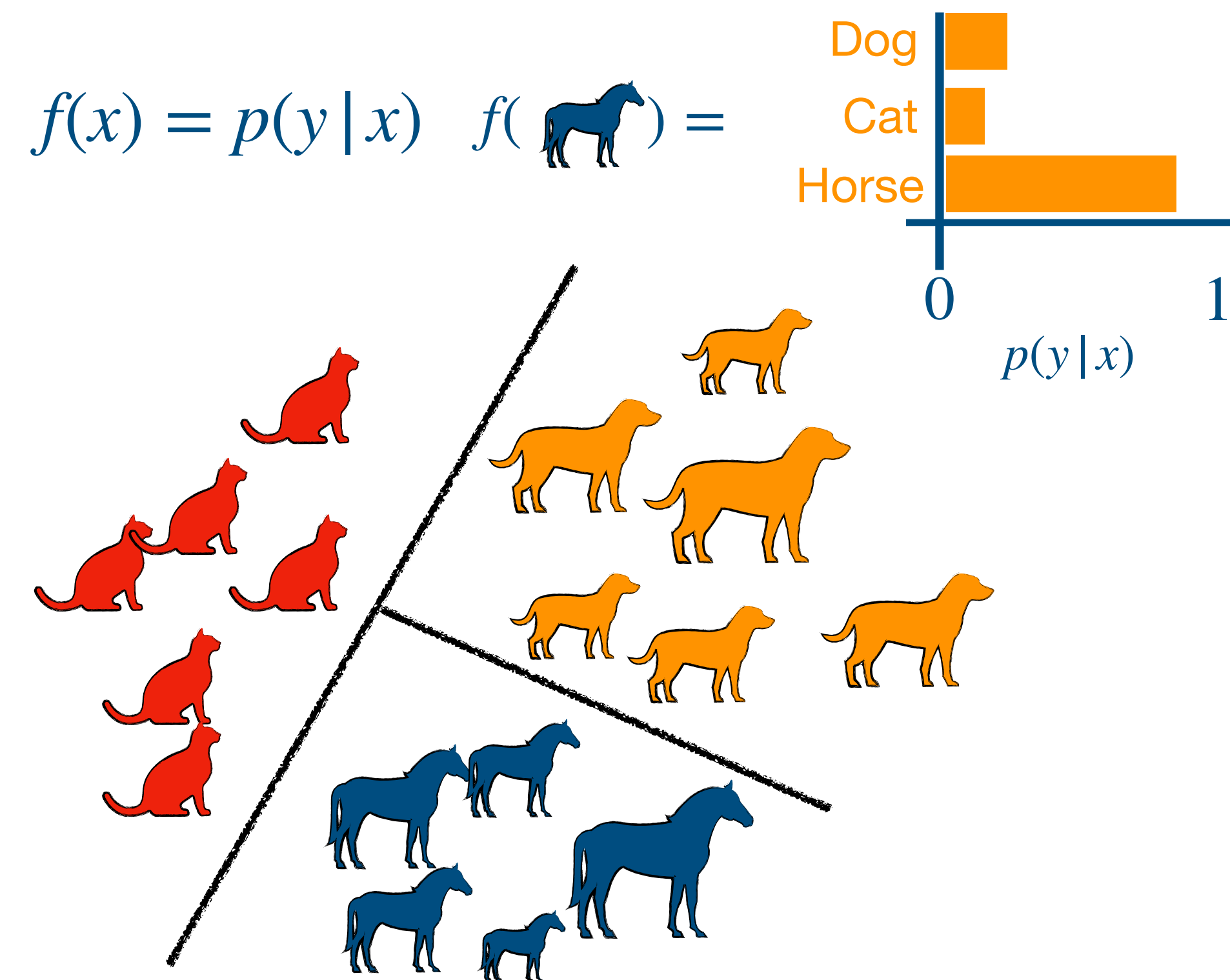
- Use the **soft-max function** $\text{softmax} : \mathbb{R}^n \rightarrow \mathbb{R}^k : \mathbb{R}^k$ is defined as follows.

- For each $s \in \mathbb{R}^k$, $\text{softmax}(s)$ is the vector $q \in \mathbb{R}^k$ such that

$$q_i = \frac{\exp(s_i)}{\sum_{j=1}^K \exp(s_j)}, \text{ for each } i = 1, 2, \dots, K$$

- The soft-max function may be seen as **converting the “scores” of K objects into a probability distribution** over the K objects, where the object having a larger score is given a larger probability.

- If $K = 2$, $s \in \mathbb{R}^2$, and $q := \text{softmax}(s)$, then $q_i = \sigma(s_1 - s_2)$



Jacobian of the soft-max function

- Lemma: suppose that $s \in \mathbb{R}^k$ and $q = \text{softmax}(s)$, then for any $i, j \in \{1, 2, \dots, K\}$:

$$\frac{\partial q_i}{\partial s_j} = q_i(\delta_{i,j} - q_j), \text{ where } \delta_{i,j} := \mathbb{I}\{i = j\}$$

- Prove this (or look it up)

Jacobian of the soft-max function

- Lemma: suppose that $s \in \mathbb{R}^k$ and $q = \text{softmax}(s)$, then for any $i, j \in \{1, 2, \dots, K\}$:

$$\frac{\partial q_i}{\partial s_j} = q_i(\delta_{i,j} - q_j), \text{ where } \delta_{i,j} := \mathbb{I}\{i = j\}$$

- Properties
 - The rows sum to zero, reflecting that softmax outputs always sum to 1.
 - The diagonal terms $q_i(1 - q_i)$ reflect how each q_i output depends on its own logit.
 - The off-diagonal terms $-q_i \cdot q_j$ show negative coupling: increasing one logit decreases the probabilities of the others.

Properties of the soft-max function

- **Shift Invariance:** For any scalar c ,

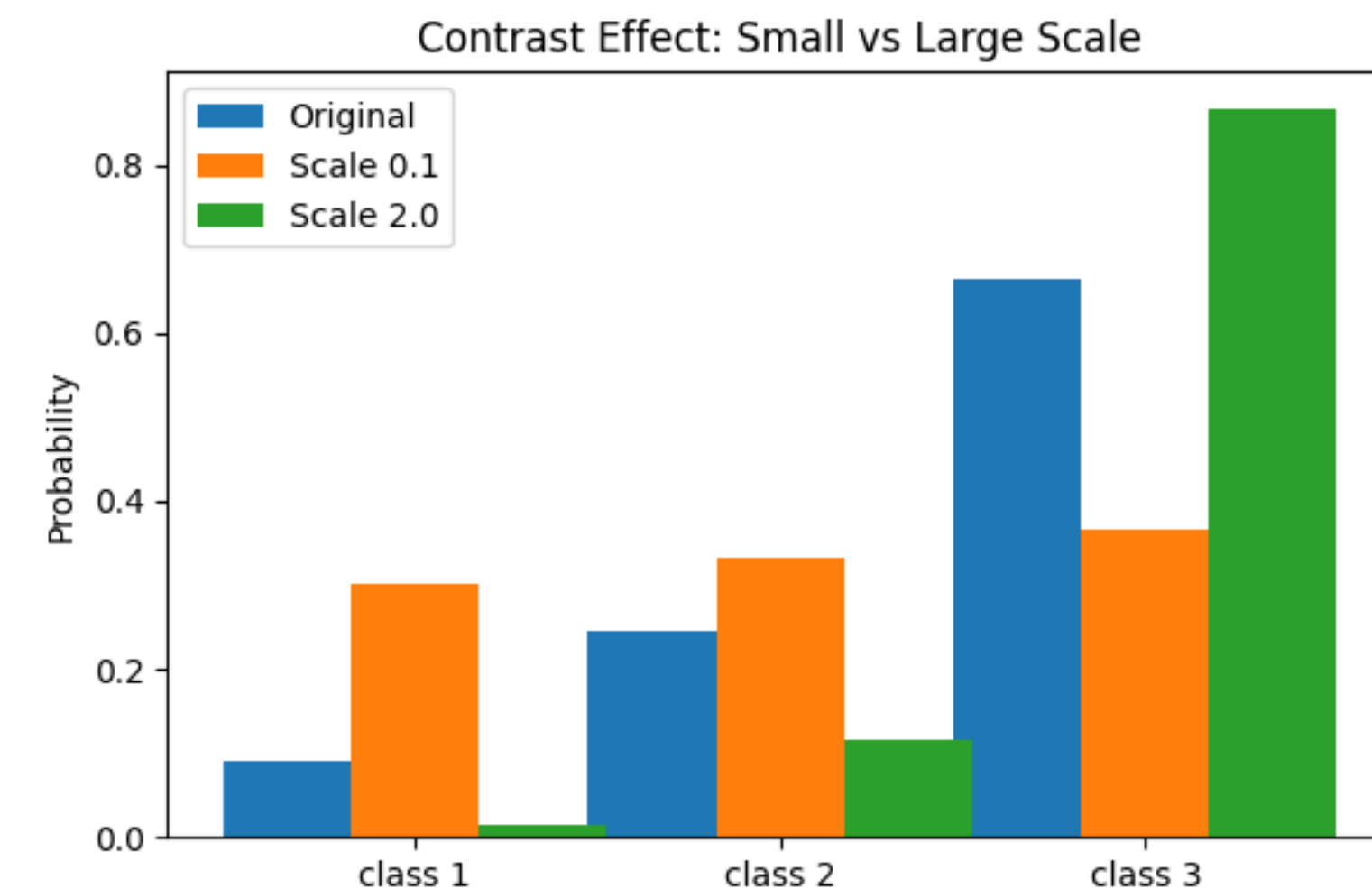
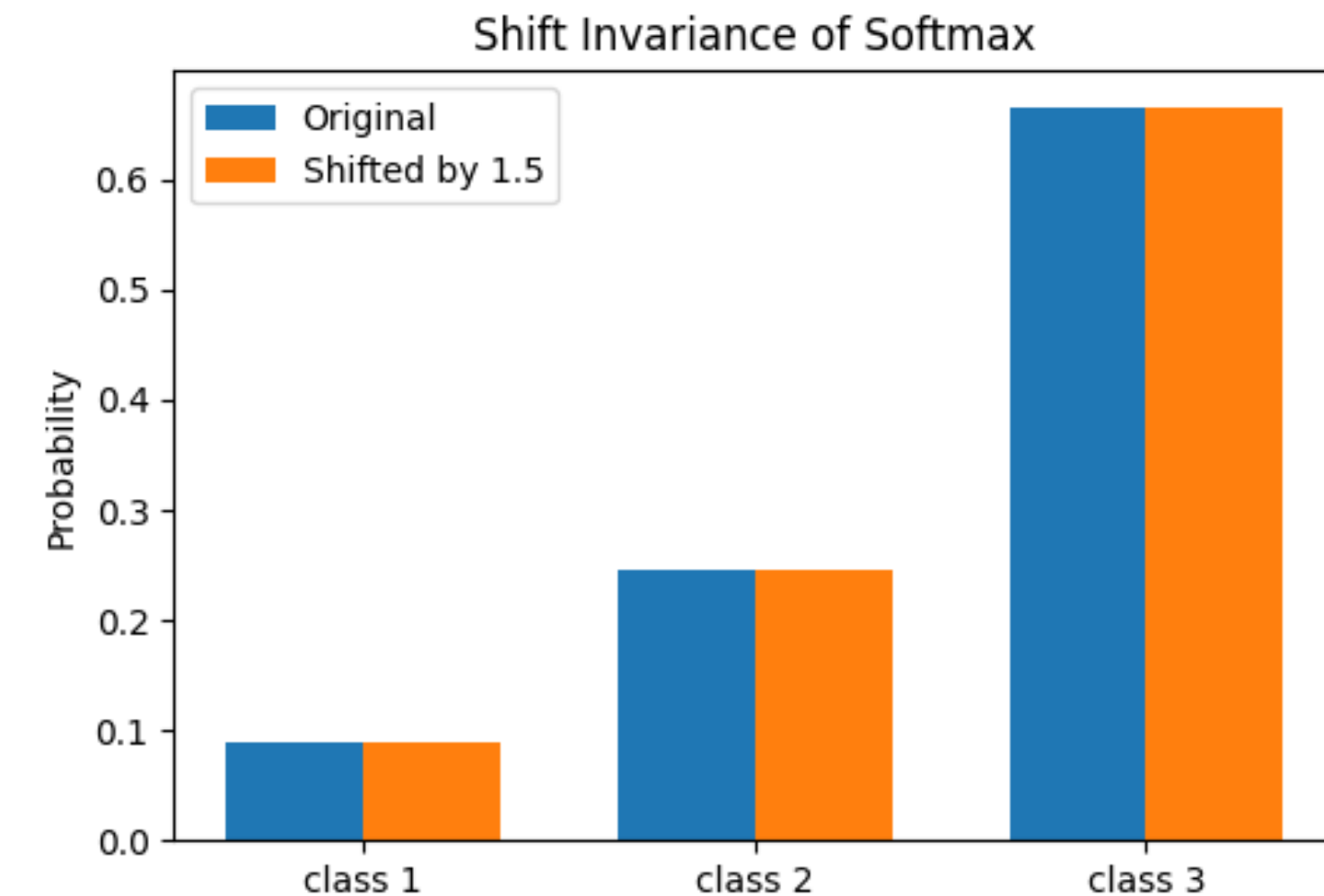
$$\text{softmax}(c + s) = \text{softmax}(s)$$

- **Scaling and Contrast:** Multiplying s by a positive scalar c changes the “contrast” of the distribution:

If $c > 1$, the softmax becomes **peakier** (closer to one-hot).

If $c < 1$, the softmax becomes **flatter** (more uniform).

- For very large c , softmax approaches a single-point mass distribution.



Predictions with Soft-Max

- Regression:

- $p_{Y|X}(\cdot | x) := \text{softmax}(Wx)$, where W is a $K \times (m + 1)$ matrix

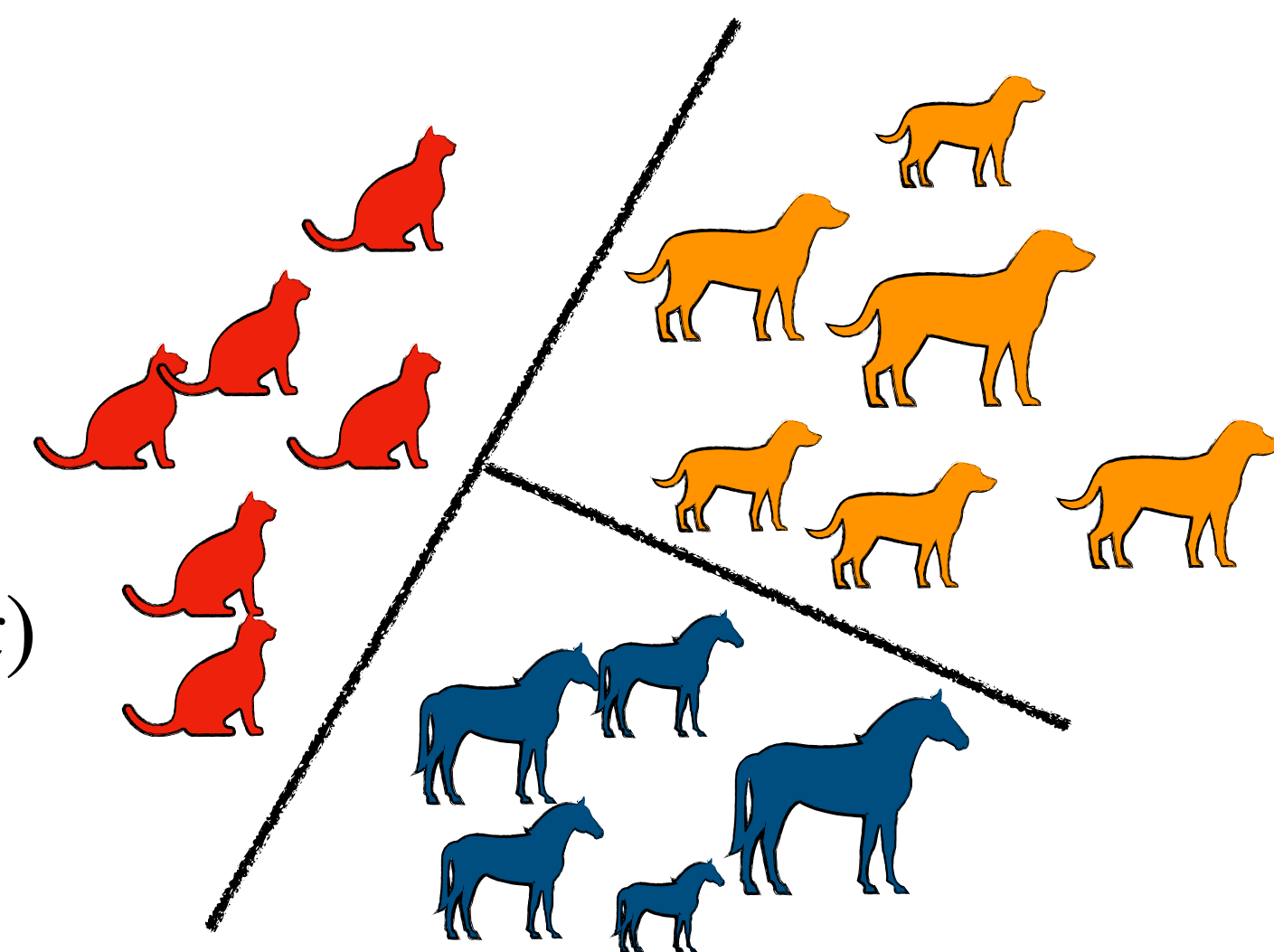
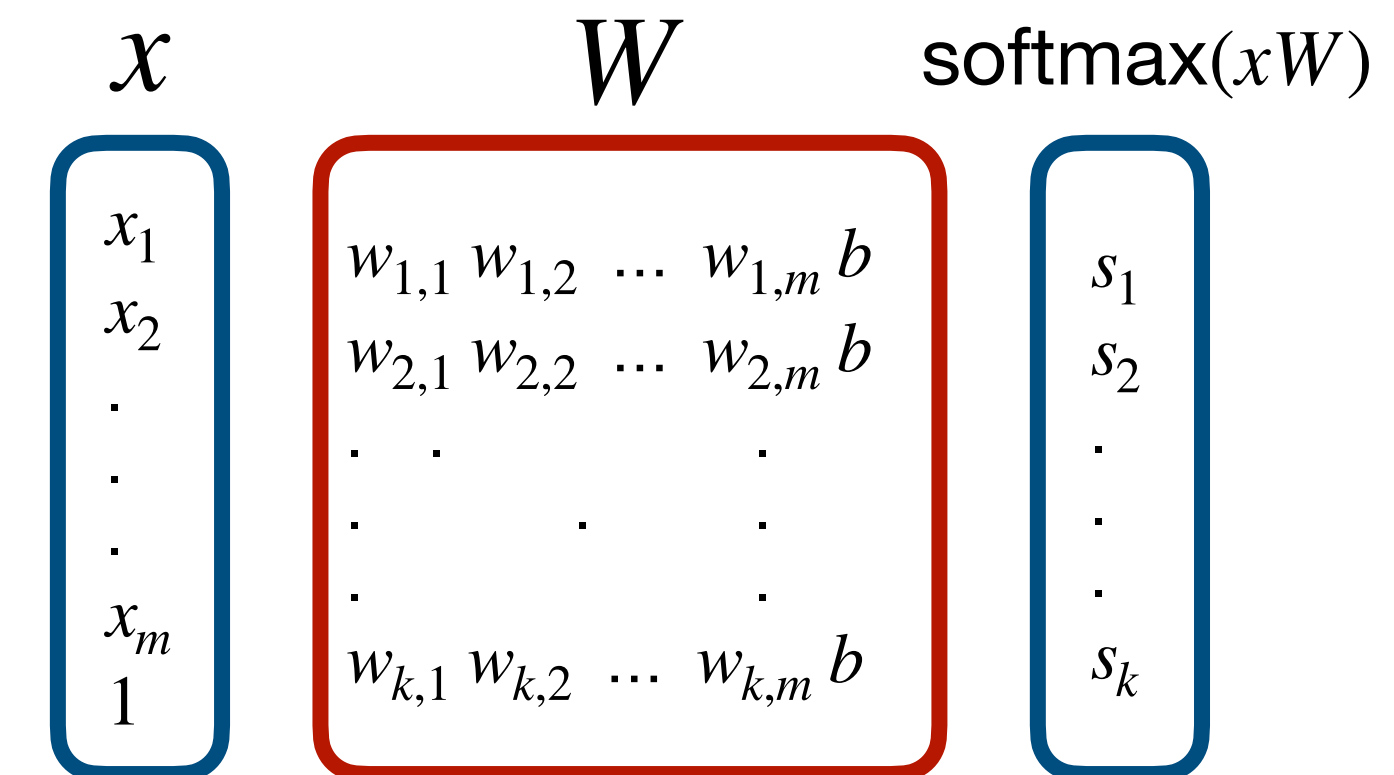
- Recall: x is an $(m + 1)$ -vector

- Optimization: find W that minimizes the cross-entropy loss by (GD or SDG)

$$L(W) := -\frac{1}{N} \sum_{i=1}^N \log_{Y|X}(y_i | x_i)$$

- Classification: for each x , declare $\arg \max_{y \in \mathcal{Y}} \hat{p}_{Y|X}(y | x)$ as the label

- Soft-max classifiers are linear classifiers: $y = f_W(x) := \arg \max_{y \in \mathcal{Y}} \hat{p}_{Y|X}(y | x)$



Artificial Neural Networks

Multi-layer Perceptron

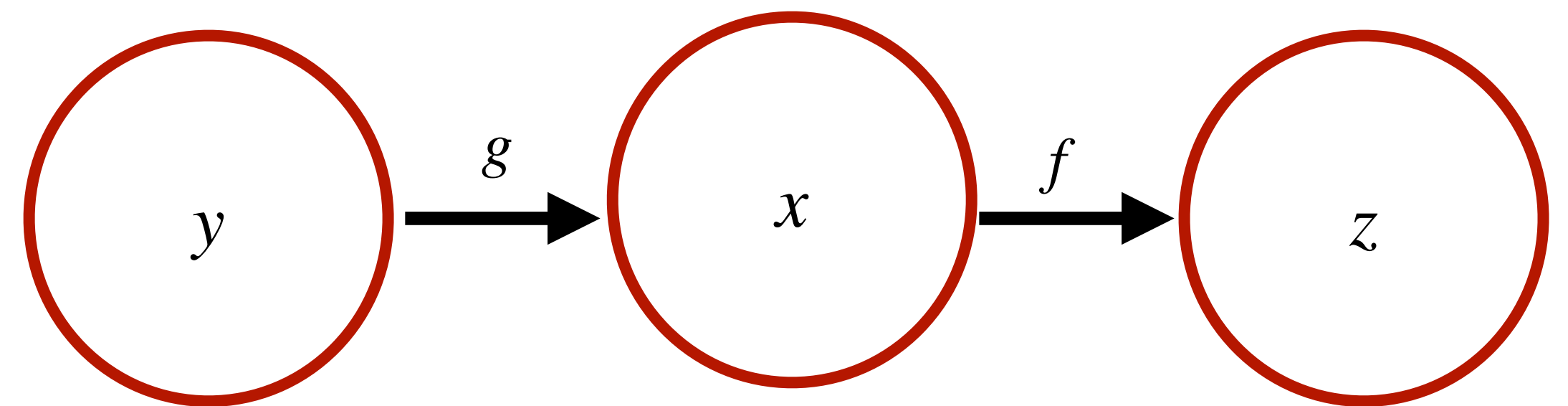
What is a Artificial NN

A computation graph representing the compositions of a set of functions

$z = f(g(y))$, where

$$f(x) = 2x + 1$$

$$g(y) = y^2 - 3$$



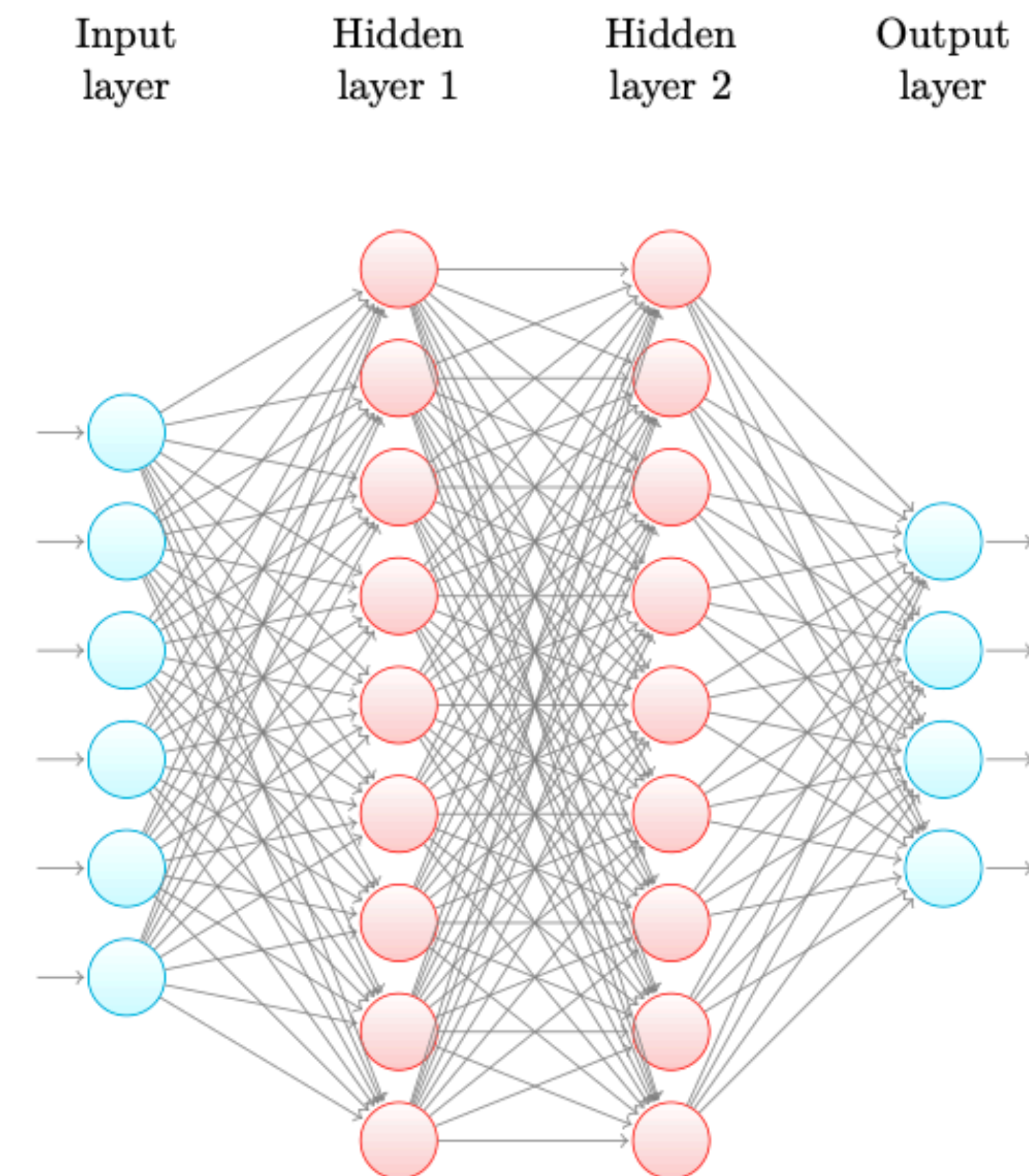
Multi-Layer Perceptron

- ANN **structured in “layers”** contains at least one “hidden layer”.

- The computation at the l^{th} hidden layer:

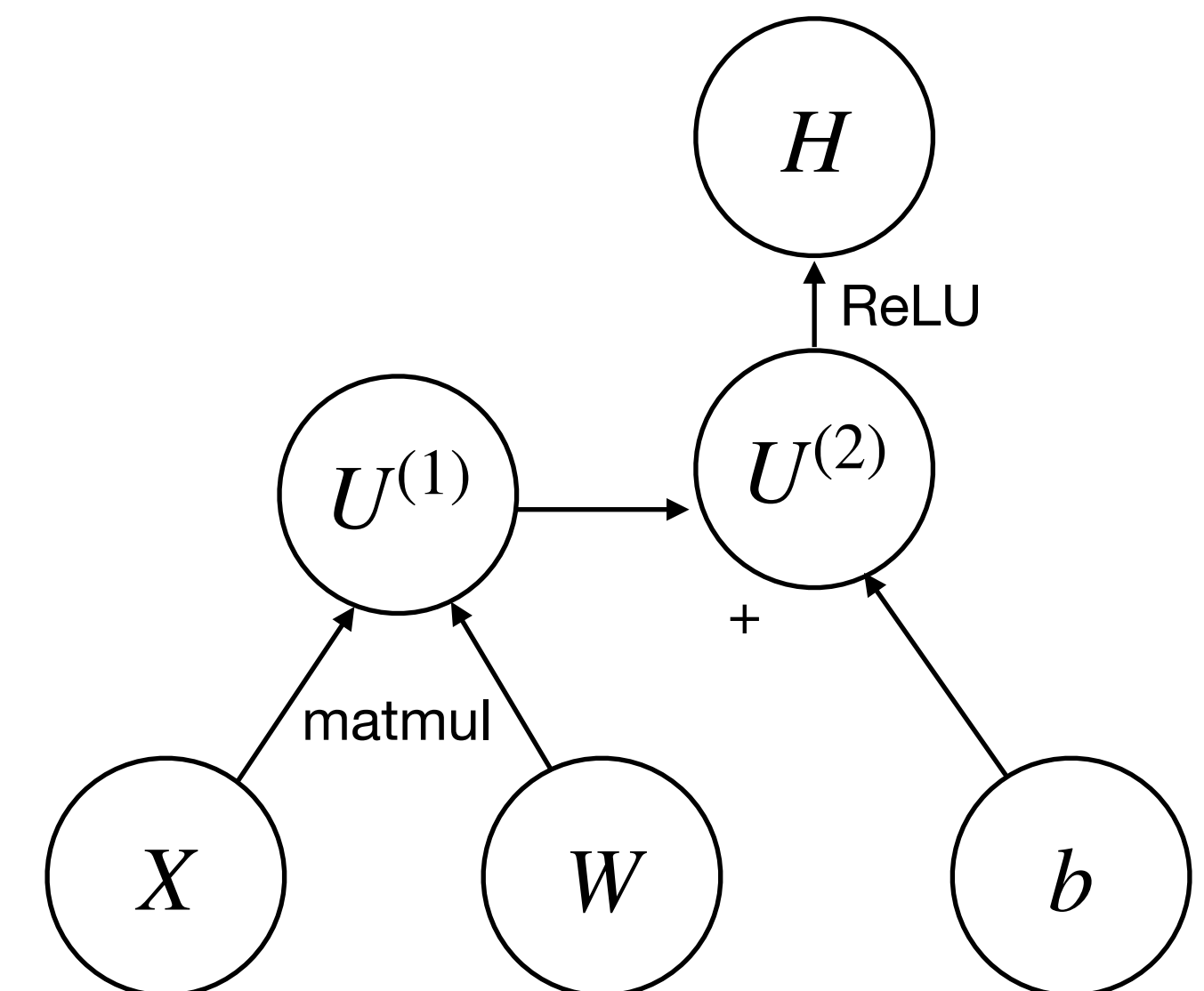
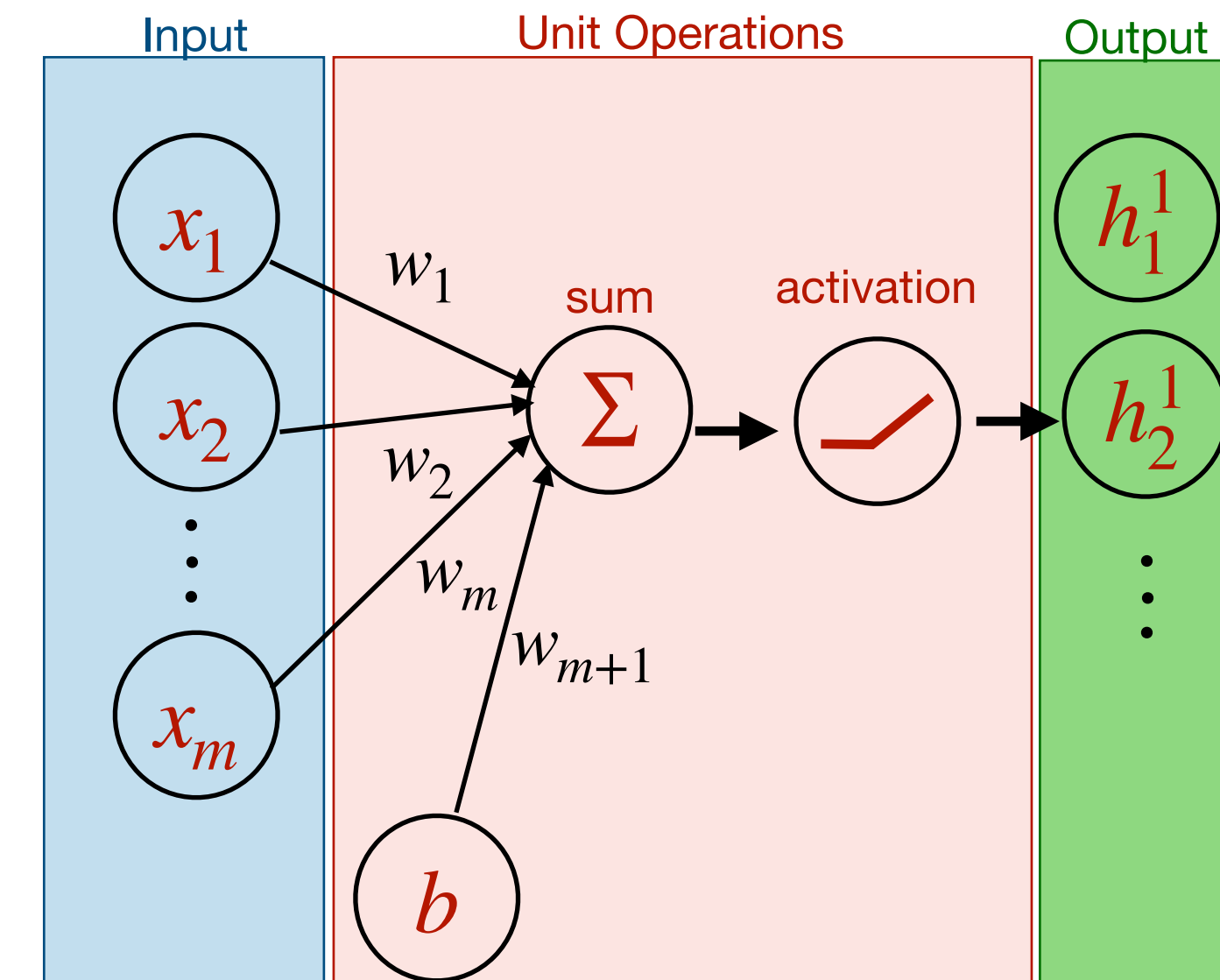
$$h^{(l)} := g(W^{(l)}h^{(l-1)}), \quad \text{where}$$

- $W^{(l)}$ is a $\dim(h^{(l)}) \times \dim(h^{(l-1)})$ **weight matrix** and
- $g : \mathbb{R} \rightarrow \mathbb{R}$ is a **nonlinear activation function**, acting on its argument vector component-wise.
- A typical “neuron” performs a **(weighted) sum followed by a nonlinear activation**.



Activation Functions

- Applied as **last operation** in a neuron / unit
- **Considerations** when selecting an activation function
 - Hidden versus output unit
 - Binary / multi-class classification, regression, generation, ...



Output Activation Functions

- **Linear function:** $\text{linear}(x) := x$

- Output in regression, auto-encoders, generative models

- **Hyperbolic tangent function:** $\tanh(x) := \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$

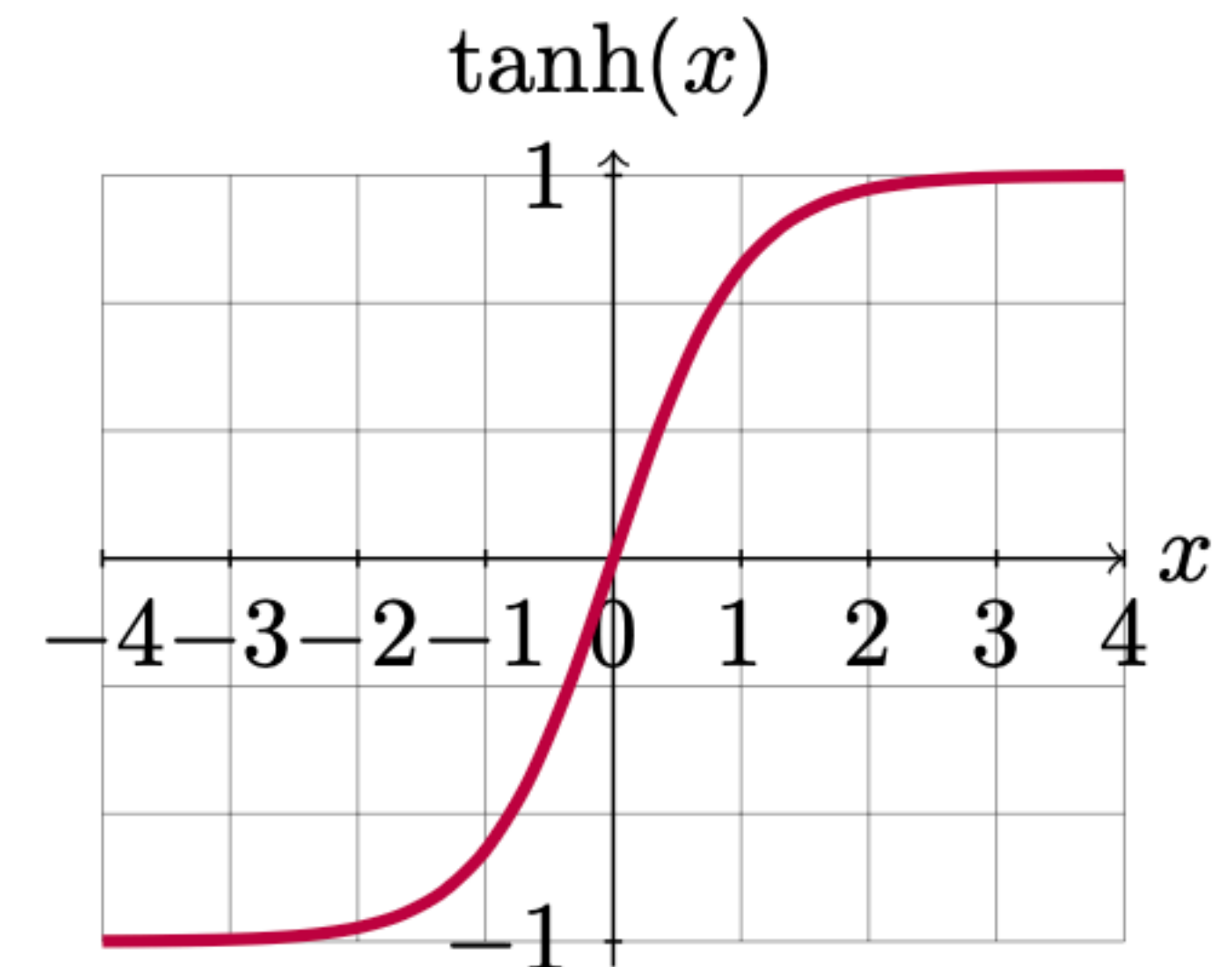
- Output unit for auto encoders or when the output needs to be in the range $(-1, 1)$

- **Sigmoid function:** $\sigma(x) := \frac{1}{1 + \exp(-x)}$

- Output unit in binary classification: NN predicts $p(y = 1 | x)$

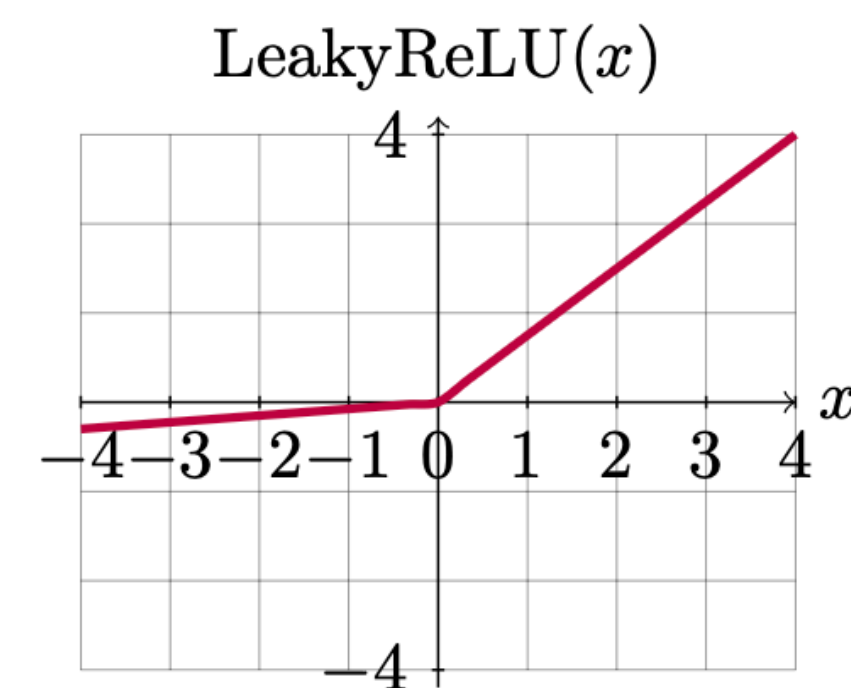
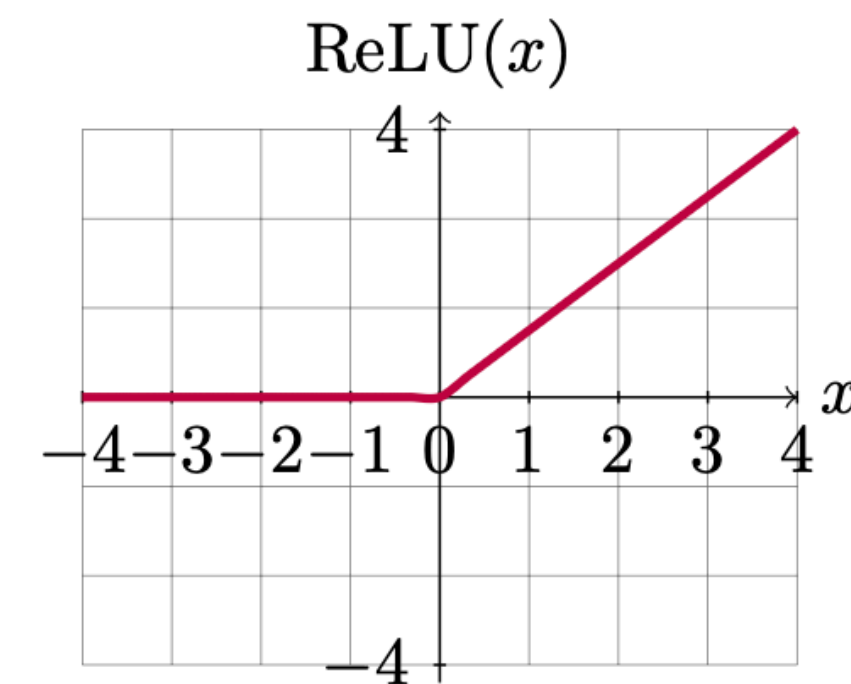
- **Softmax function:** $\text{softmax}(x) := \frac{\exp(x_i)}{\sum_{j=1}^K \exp(x_j)}$

- Output unit in multi-class classification: NN predicts class conditional probability $p(y = c | x)$ for $c \in C$



Hidden Activation Functions

- **Rectified Linear Unit:** $\text{ReLU} := \max\{x, 0\}$
 - Generally use this! It is simple, efficient, avoids vanishing gradients for positive inputs
- **Generalization of ReLU:** $g(x) := \max\{x, 0\} + \alpha \min\{x, 0\}$
 - Absolute value function: $\alpha = -1$. Object recognition - encourages invariance to polarity reversal of input illumination
 - Leaky ReLU: very small α . Fixes “dying ReLU” (no gradient when output is zero) problem by allowing small negative outputs.
 - Parametric ReLU (PReLU): learnable α
- **Gaussian Error Linear Unit:** $\text{GELU}(x) = x \cdot \phi(x)$, where cumulative distribution function of the standard normal distribution
 - Common in transformer-based models (e.g., BERT, GPT)



MLP Classification Model

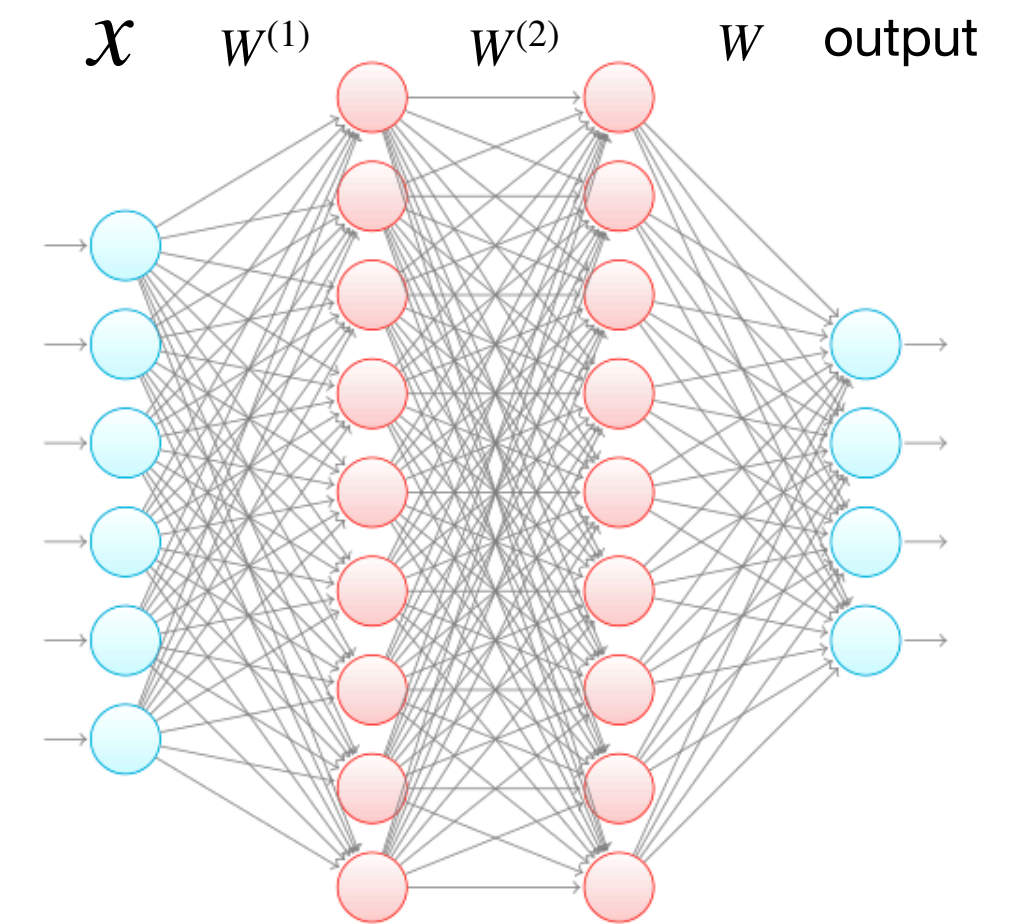
- **MLP with two hidden layers**

$$p_{Y|X}(\cdot | x) := \text{softmax} \left(W \cdot g \left(W^{(2)} \cdot g \left(W^{(1)} x \right) \right) \right), \text{ where}$$

$W^{(1)}$, $W^{(2)}$ and W are weights for the first, second and output layers

$g(\cdot)$ is the activation function

- We will **use SGD on the cross-entropy loss to learn** the parameters $W^{(1)}$, $W^{(2)}$ and W .
- The **obtained $\hat{p}_{Y|X}$** is then used for classification.



Universal Approximation Theorem

Universal Approximation Theorem (UAT)

- UAT is a **foundational result in neural network theory** (Cybenko, 89; Hornik, 91).
- **An ANN with one hidden layer can approximate any continuous function on a compact domain, arbitrarily well.**
- Justifies the expressive capacity of ANNs
- Explains why even shallow networks can model complex nonlinear functions.

Universal Approximation Theorem (UAT)

- Setup
 - Let $I^n = [0,1]^n \subset \mathbb{R}^n$ be the compact domain.
 - Let $C(I^n)$ denote the space of all continuous real-valued functions on I^n .
 - Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous, bounded, non-constant, monotonically increasing activation function (e.g., a sigmoid).
 - Let $f : I^n \rightarrow \mathbb{R}$ be any continuous function we want to approximate.

Universal Approximation Theorem (UAT)

- Then for any $f \in C(I^n)$ and any $\varepsilon > 0$, there exist:
 - An integer N
 - Weight vectors $w_1, \dots, w_N \in \mathbb{R}^n$
 - Biases $\theta_1, \dots, \theta_N \in \mathbb{R}$
 - Output weights $\alpha_1, \dots, \alpha_N \in \mathbb{R}$
 - such that the function

$$F_N(x) = \sum_{k=1}^N \alpha_k \sigma(w_k^T x + \theta_k) \text{ satisfies } |f(x) - F_N(x)| < \varepsilon \quad \text{for all } x \in I^n.$$

Universal Approximation Theorem (UAT)

- Then for any $f \in C(I^n)$ and any $\varepsilon > 0$, there exist:

- An integer N
- Weight vectors $w_1, \dots, w_N \in \mathbb{R}^n$
- Biases $\theta_1, \dots, \theta_N \in \mathbb{R}$
- Output weights $\alpha_1, \dots, \alpha_N \in \mathbb{R}$
- such that the function

Note: This is exactly the structure of a single-hidden-layer neural network with N hidden units.

• $F_N(x) = \sum_{k=1}^N \alpha_k \sigma(w_k^T x + \theta_k)$ satisfies $|f(x) - F_N(x)| < \varepsilon$ for all $x \in I^n$.

Universal Approximation Theorem (UAT)

- Interpretation
 - A **single-hidden-layer neural network with a suitable activation function can approximate any continuous function** on a compact set, to arbitrary accuracy, if the network is allowed to be large enough
- Note:
 - It **does not specify how large N must be.**
 - It **does not give a constructive way to find the weights.**
 - It **does not guarantee that gradient descent will find f .**

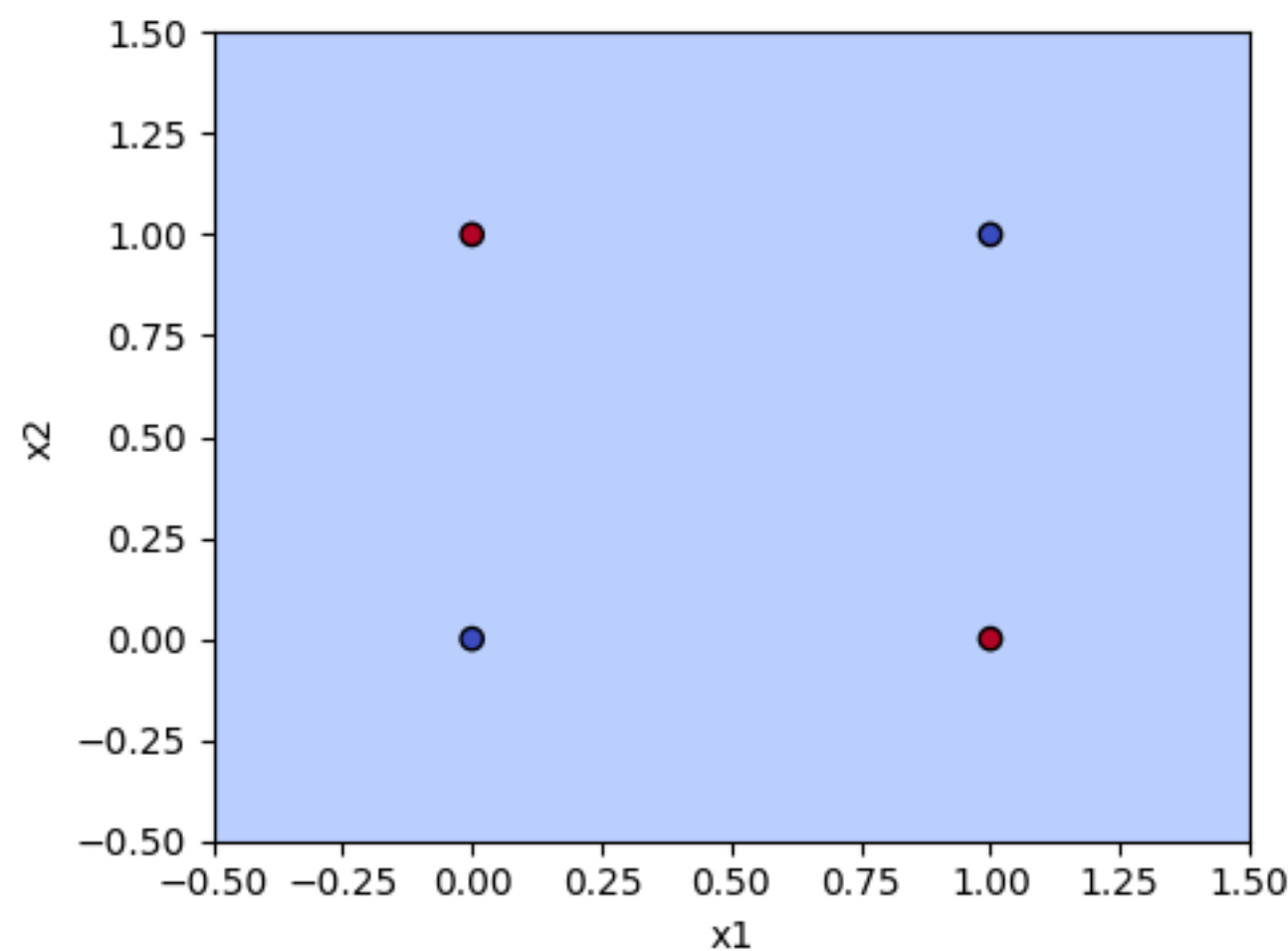
Universal Approximation Theorem (UAT)

- The theorem suggests that **the function exists** not that you will find it
 - Shallow nets **may need (exponentially) more width**
 - Shallow nets **may overfit**

Universal Approximation Theorem (UAT)

- An example at work: XOR problem

Draw a hyperplane that
separates the **red points**
from the **blue points**

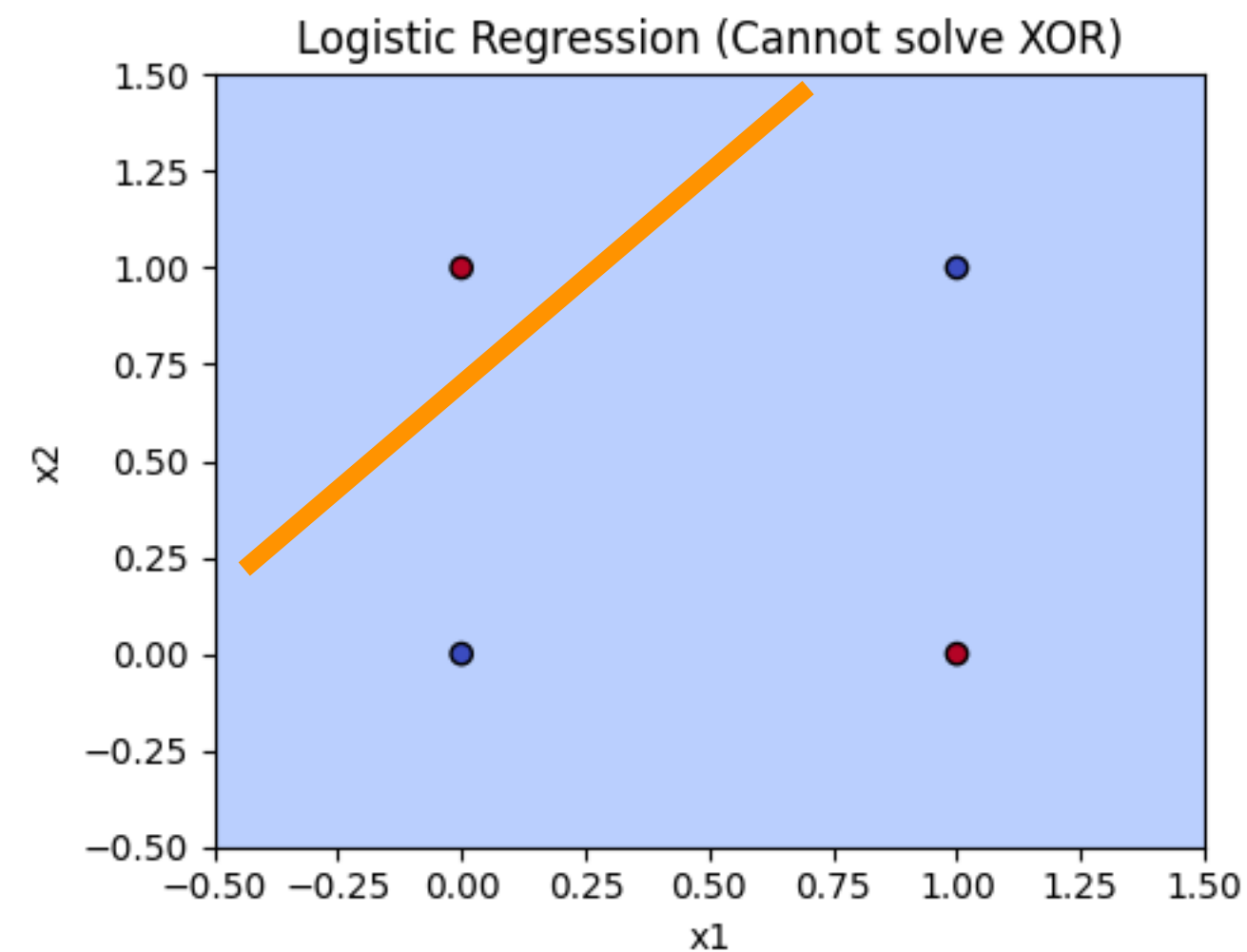


Universal Approximation Theorem (UAT)

- An example at work: XOR problem

No linear classifier that
can separate the two
classes

Logistic-regression like network
won't work

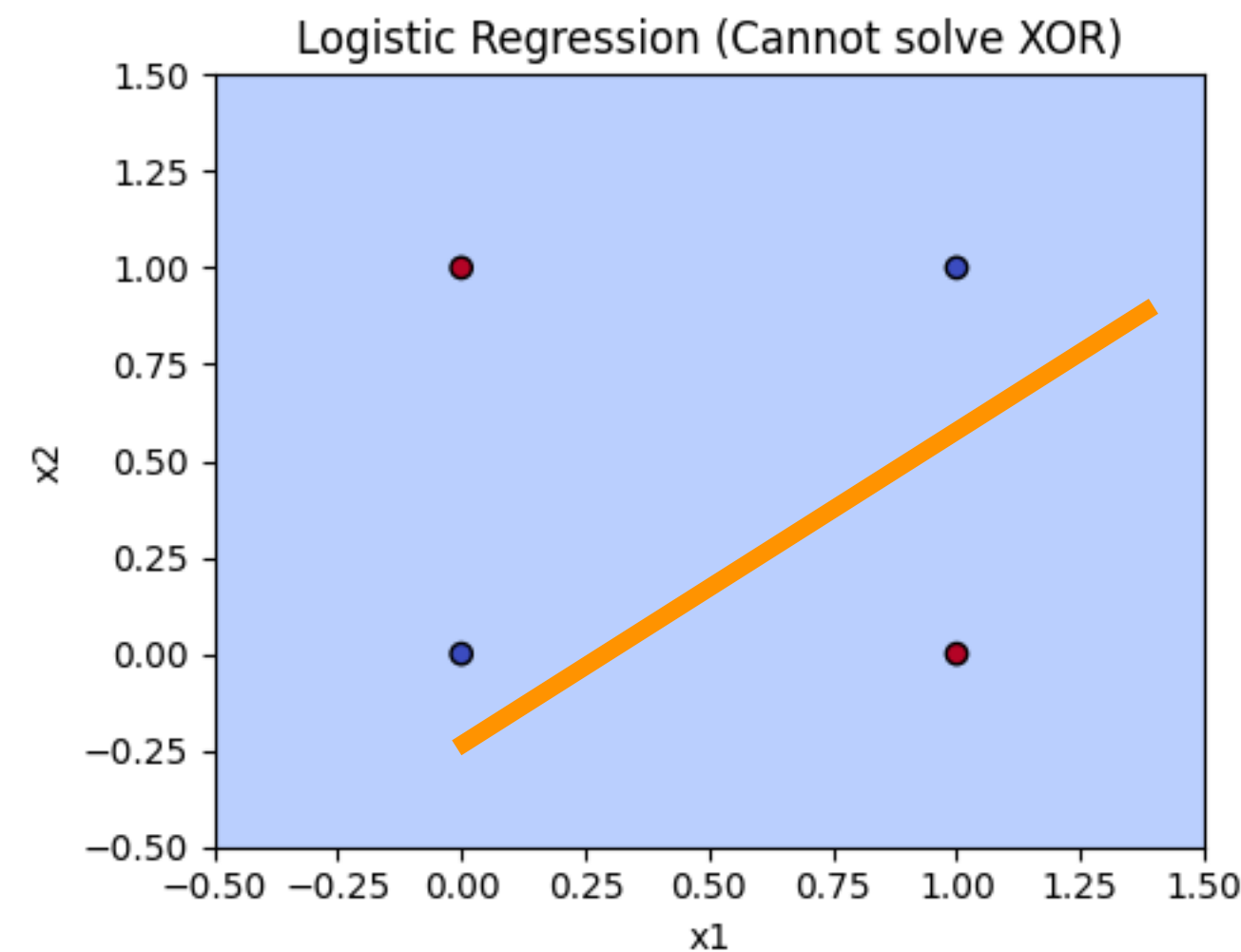


Universal Approximation Theorem (UAT)

- An example at work: XOR problem

No linear classifier that
can separate the two
classes

Logistic-regression like network
won't work

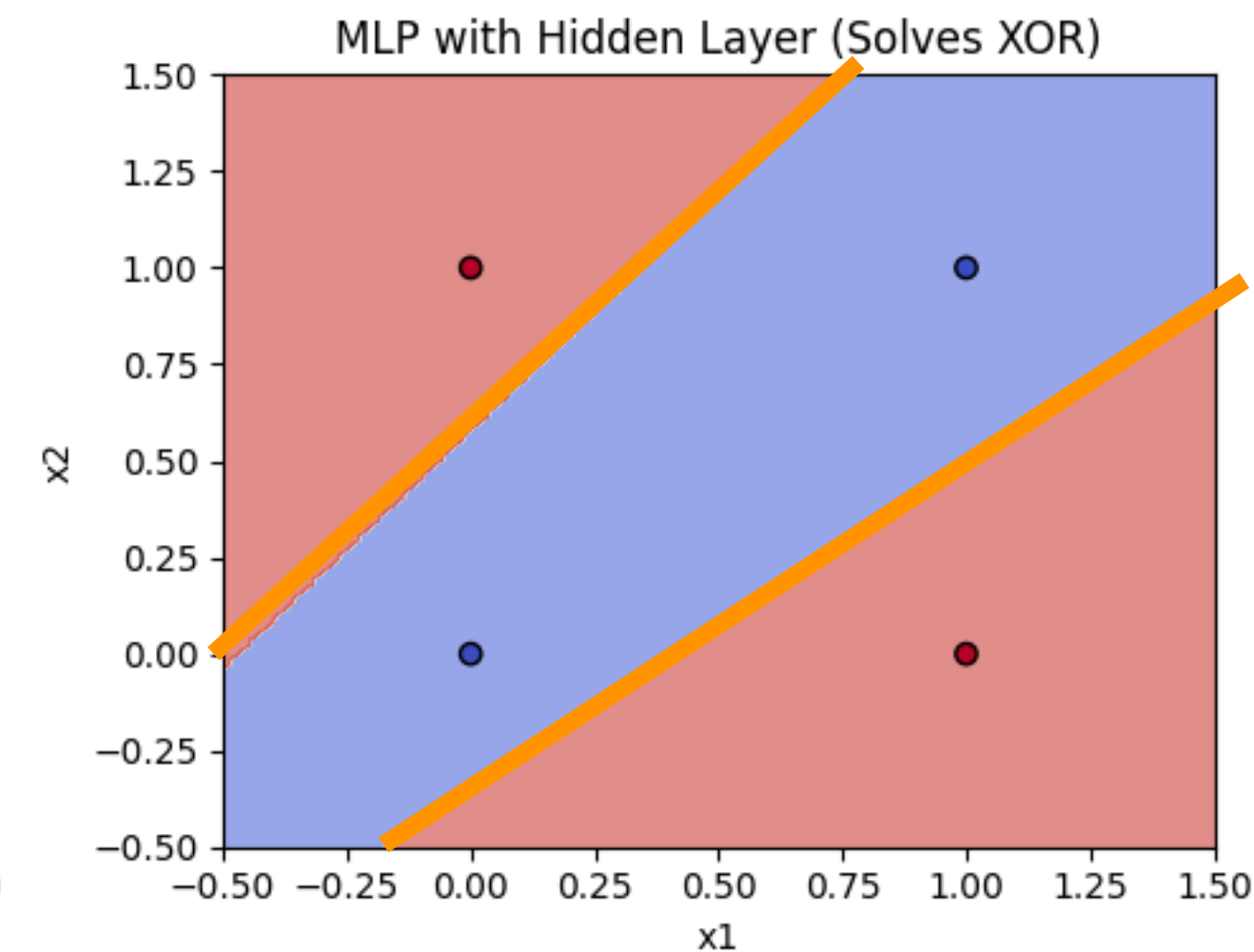
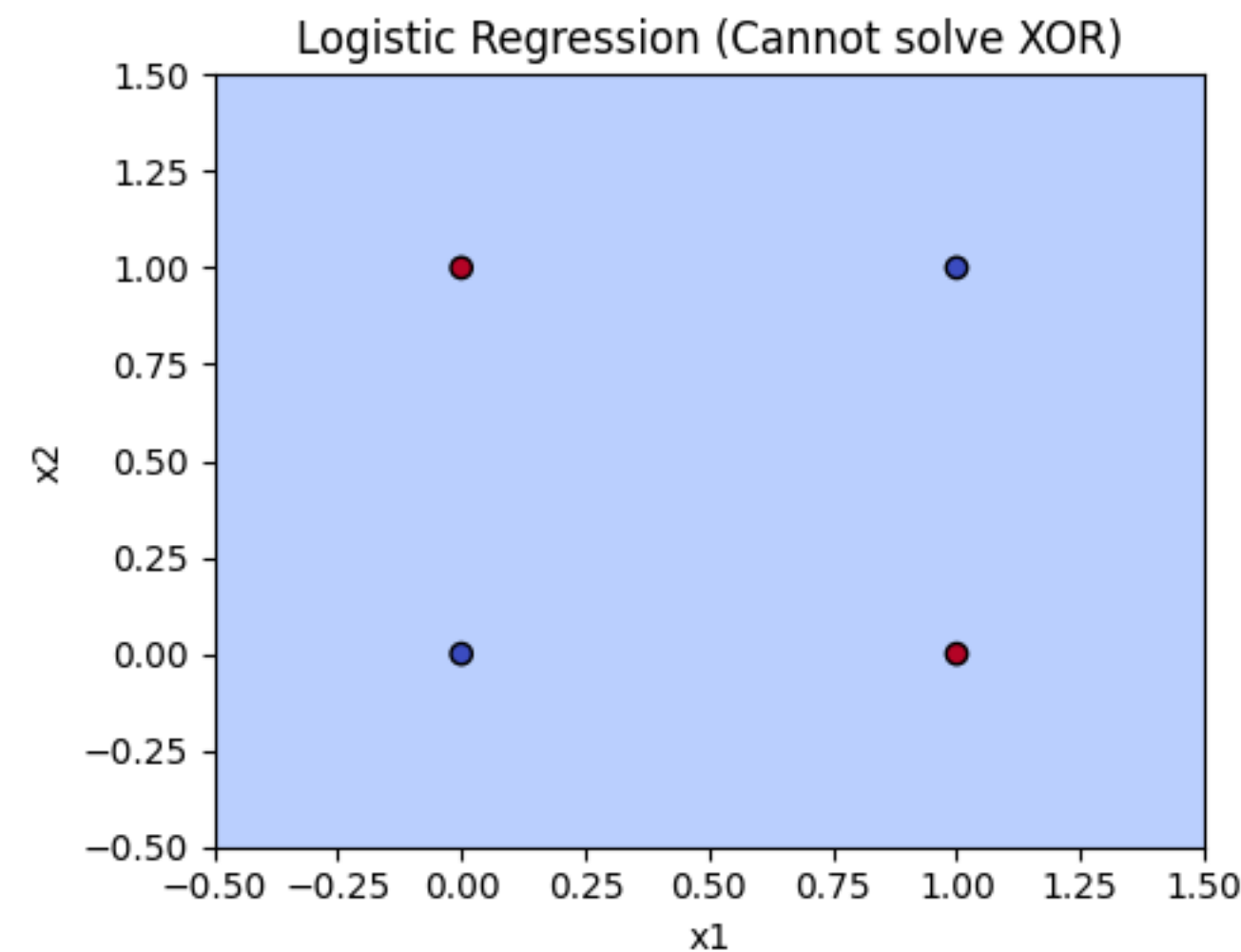


Universal Approximation Theorem (UAT)

- An example at work: XOR problem

No linear classifier that
can separate the two
classes

Logistic-regression like network
won't work

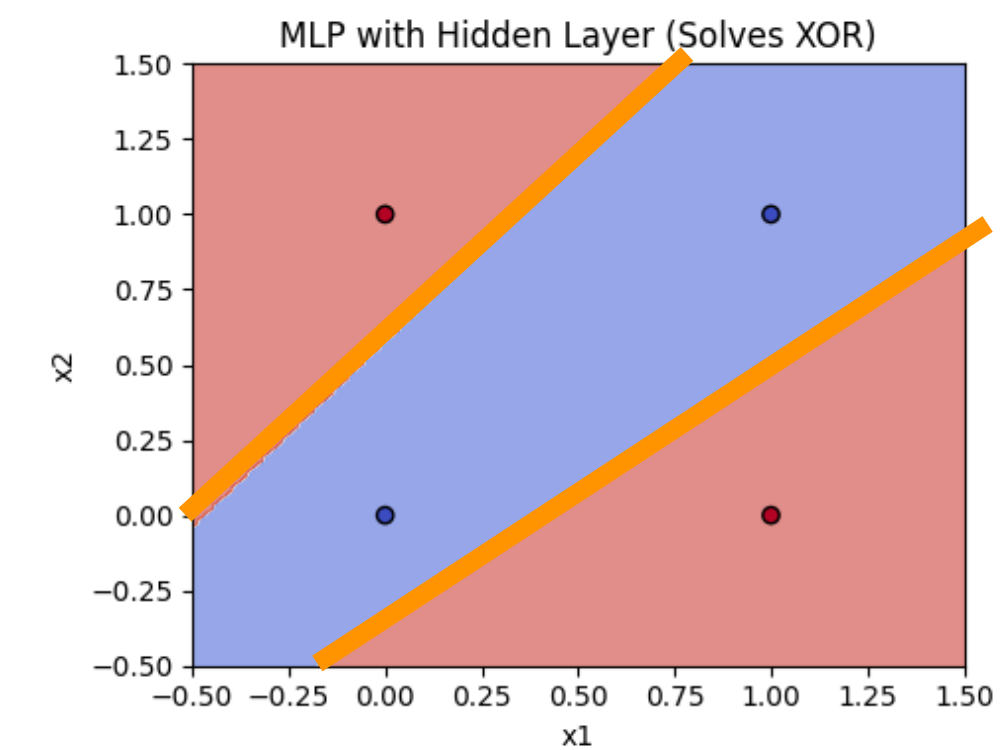
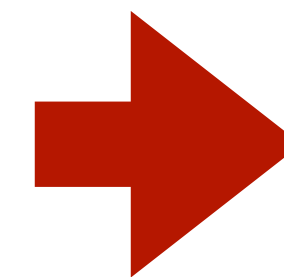
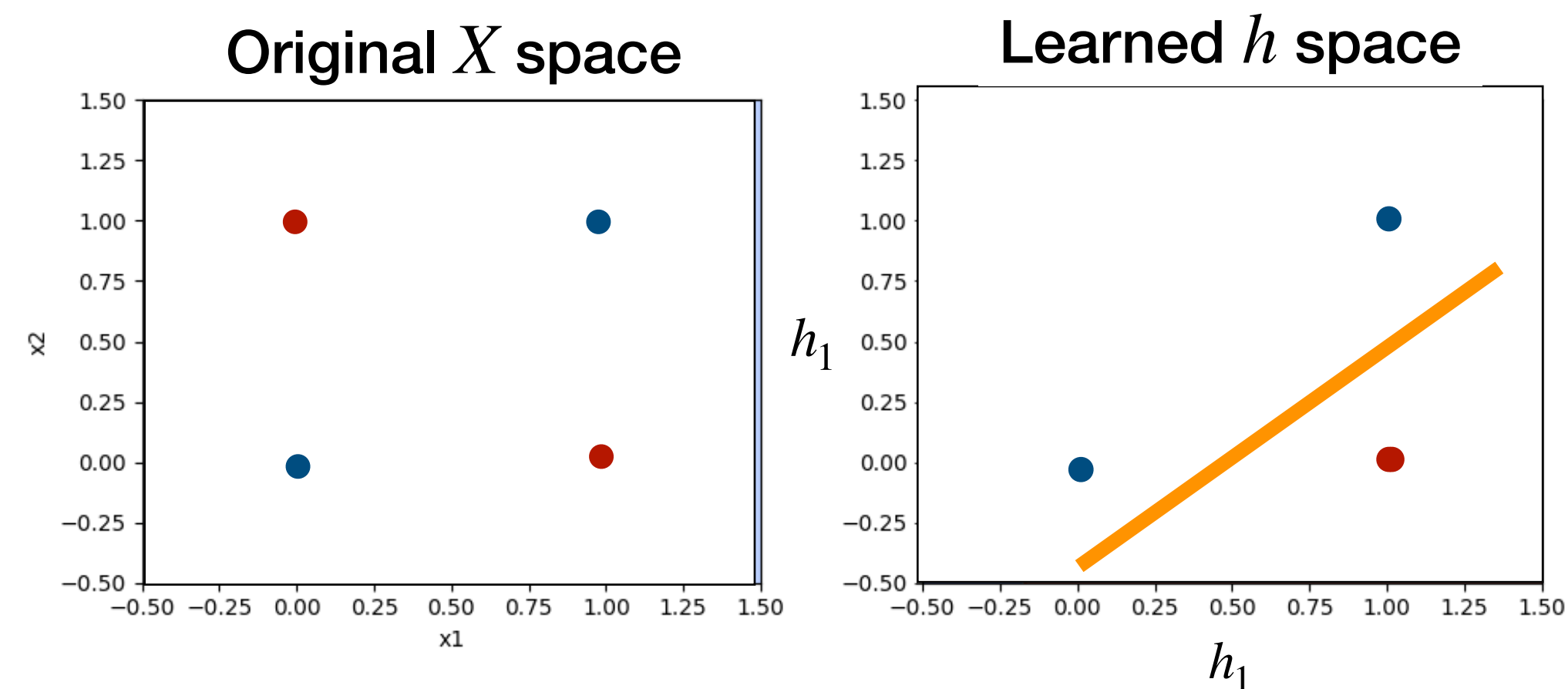


MLP with single hidden
layer of dimension 2
works

Universal Approximation Theorem (UAT)

- What is the MLP doing that logistic regression can't do?

∴



See also Goodfellow Book Section 6.1 for a ReLU-activated MLP construction

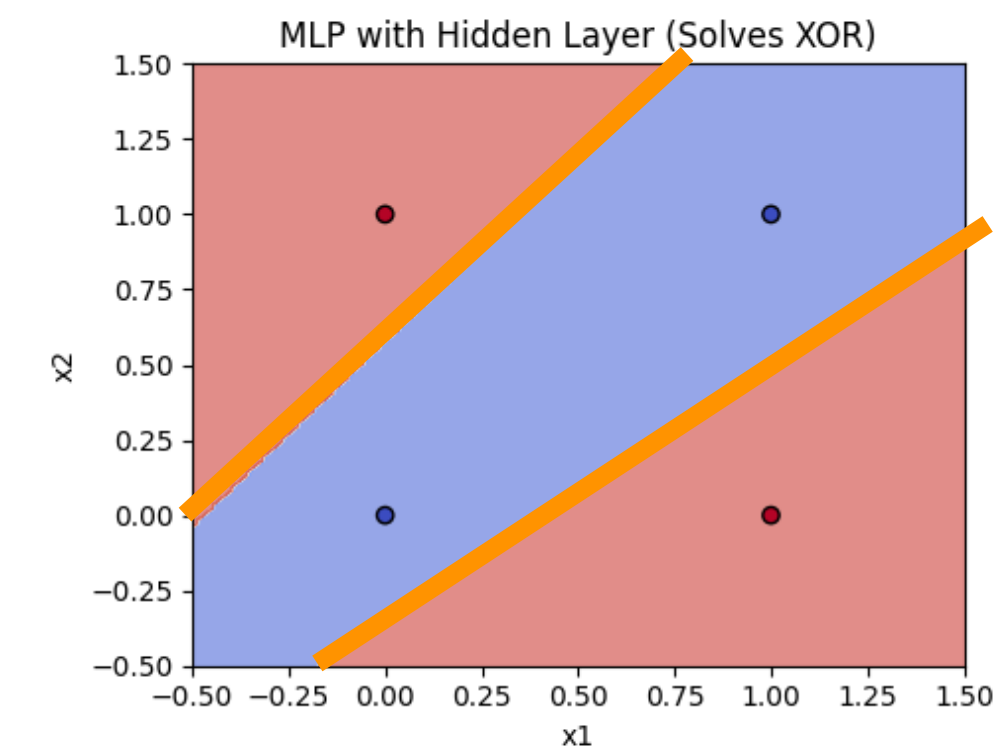
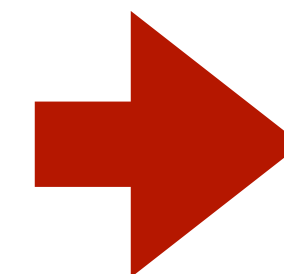
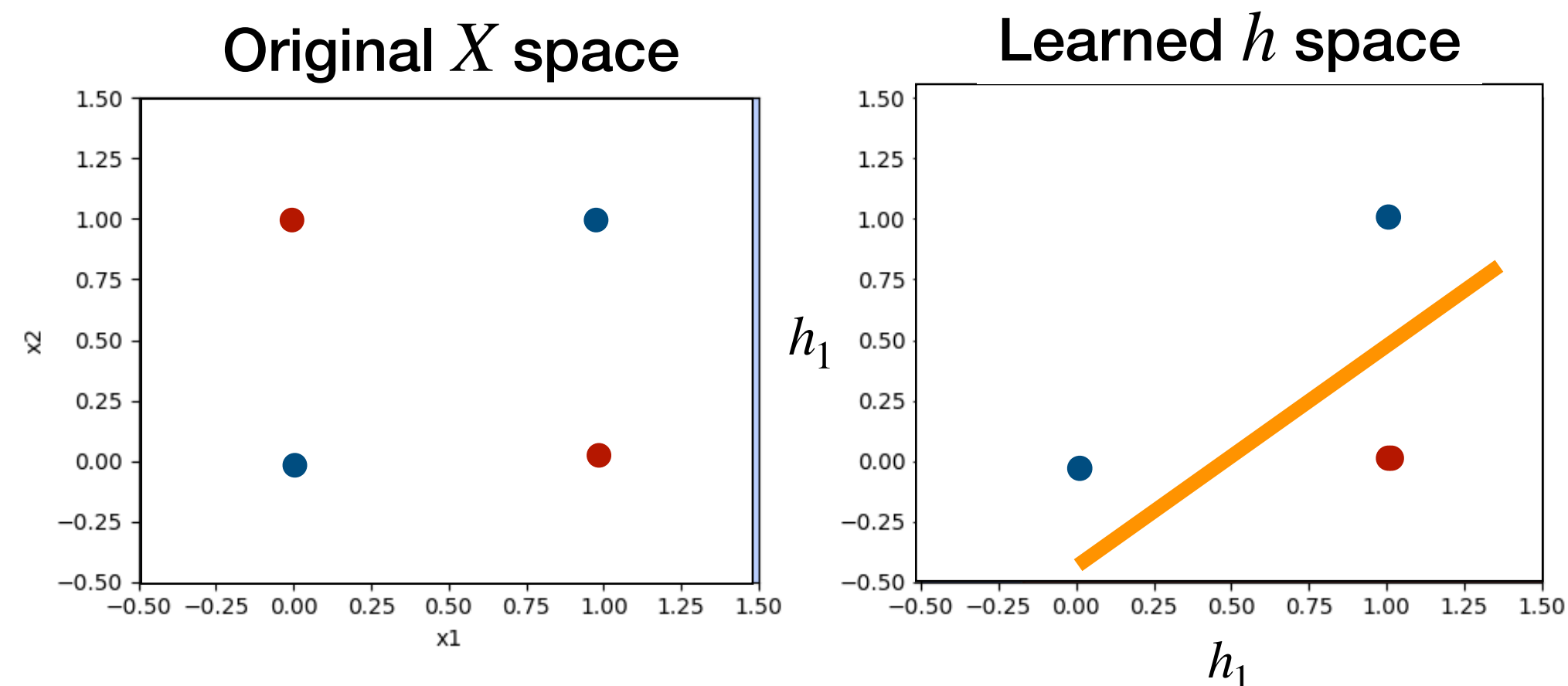
Universal Approximation Theorem (UAT)

- Consider an **MLP** with a **single hidden layer**, **sigmoid hidden activation** and **linear output activation**

$$h^{(1)} := \sigma(W^{(1)}x) \quad // \text{ we have 2-hidden units - } h \text{ is 2-D so we can easily plot this}$$

$$o := \text{linear}(Wh^{(1)})$$

- $h^{(1)} := \sigma(W^{(1)}x)$ maps the linearly in-separable XOR input to a new linearly separable feature-space in $\mathbb{R}^2$



See also Goodfellow Book Section 6.1 for a ReLU-activated MLP construction

Universal Approximation Theorem (UAT)

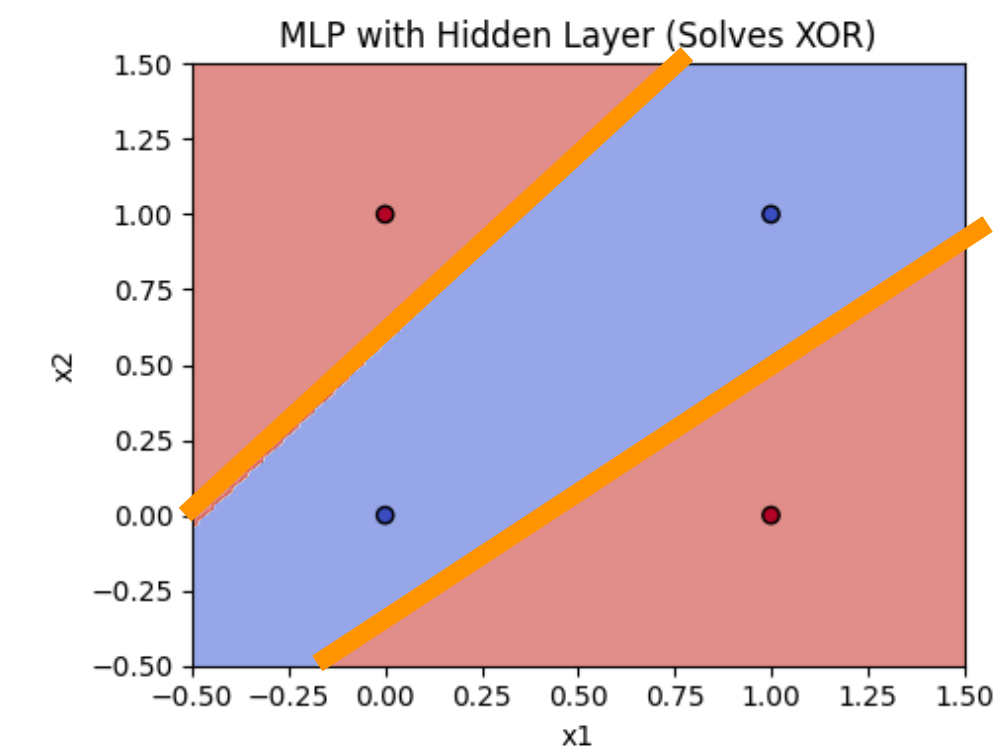
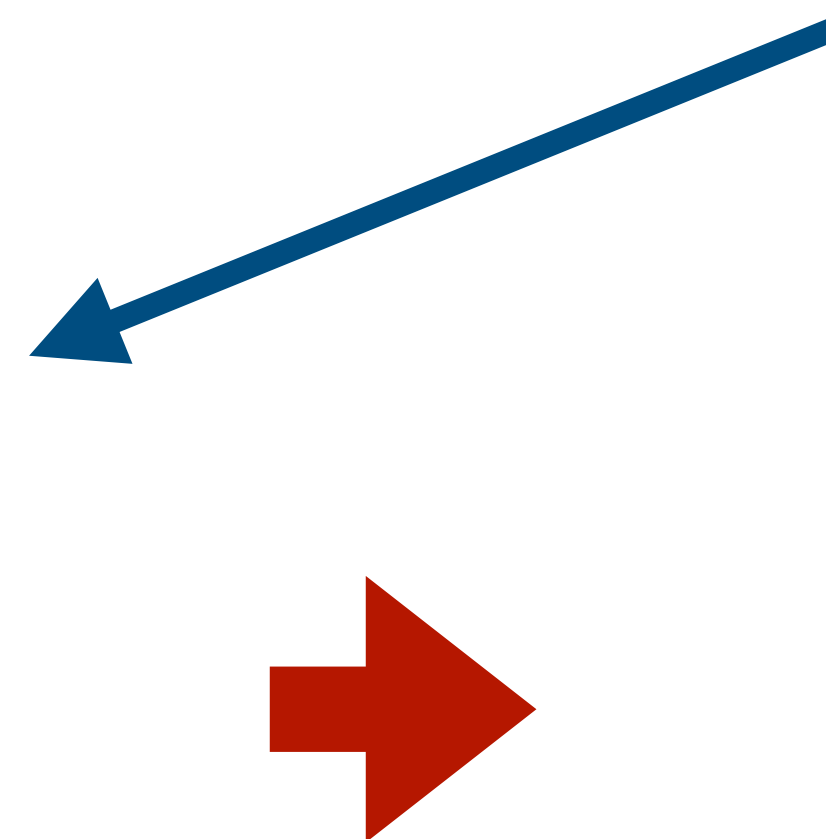
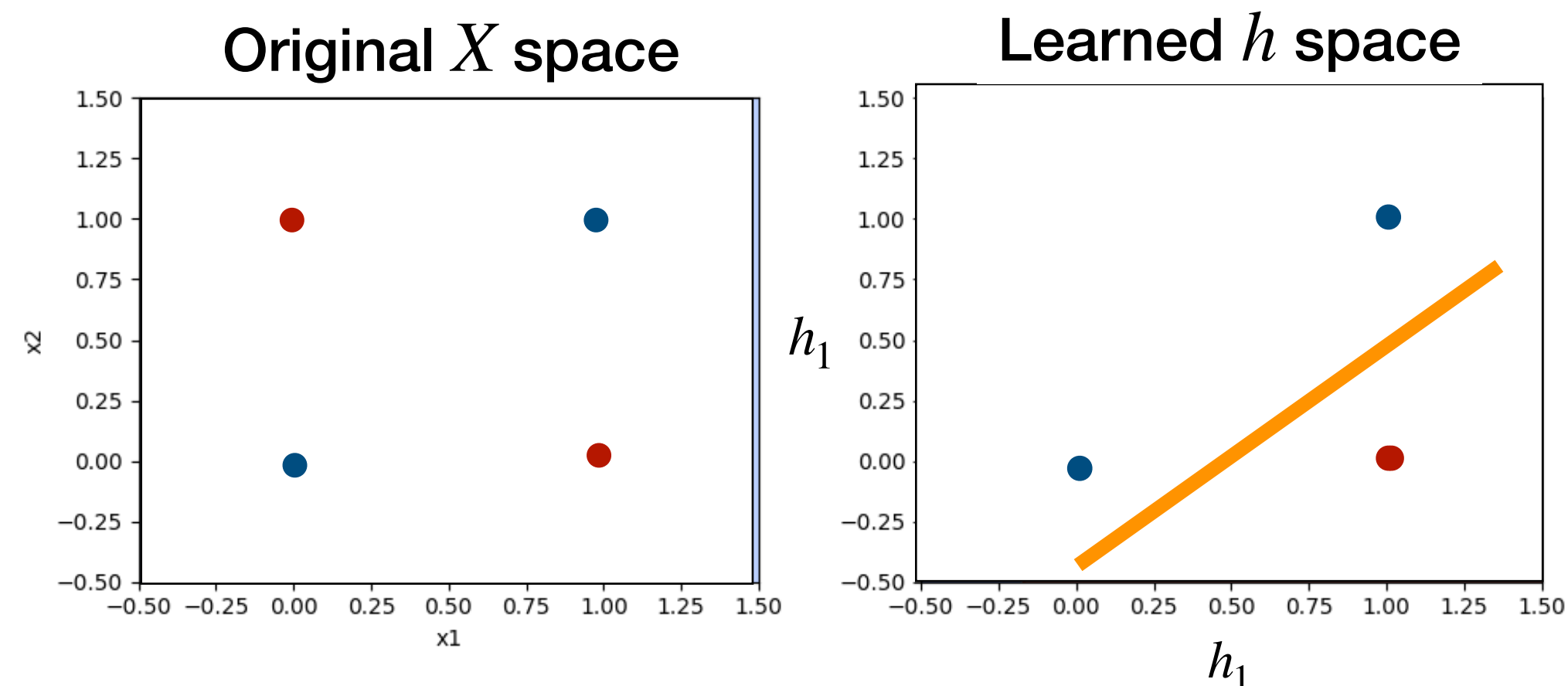
- Consider an **MLP** with a **single hidden layer**, **sigmoid hidden activation** and **linear output activation**

$$h^{(1)} := \sigma(W^{(1)}x)$$

$$o := \text{linear}(Wh^{(1)})$$

- $h^{(1)} := \sigma(W^{(1)}x)$ maps the linearly in-separable XOR input to a new linearly separable feature-space in $\mathbb{R}^2$

Learning good feature spaces is a fundamental capability of ANNs, which most other ML method do not have



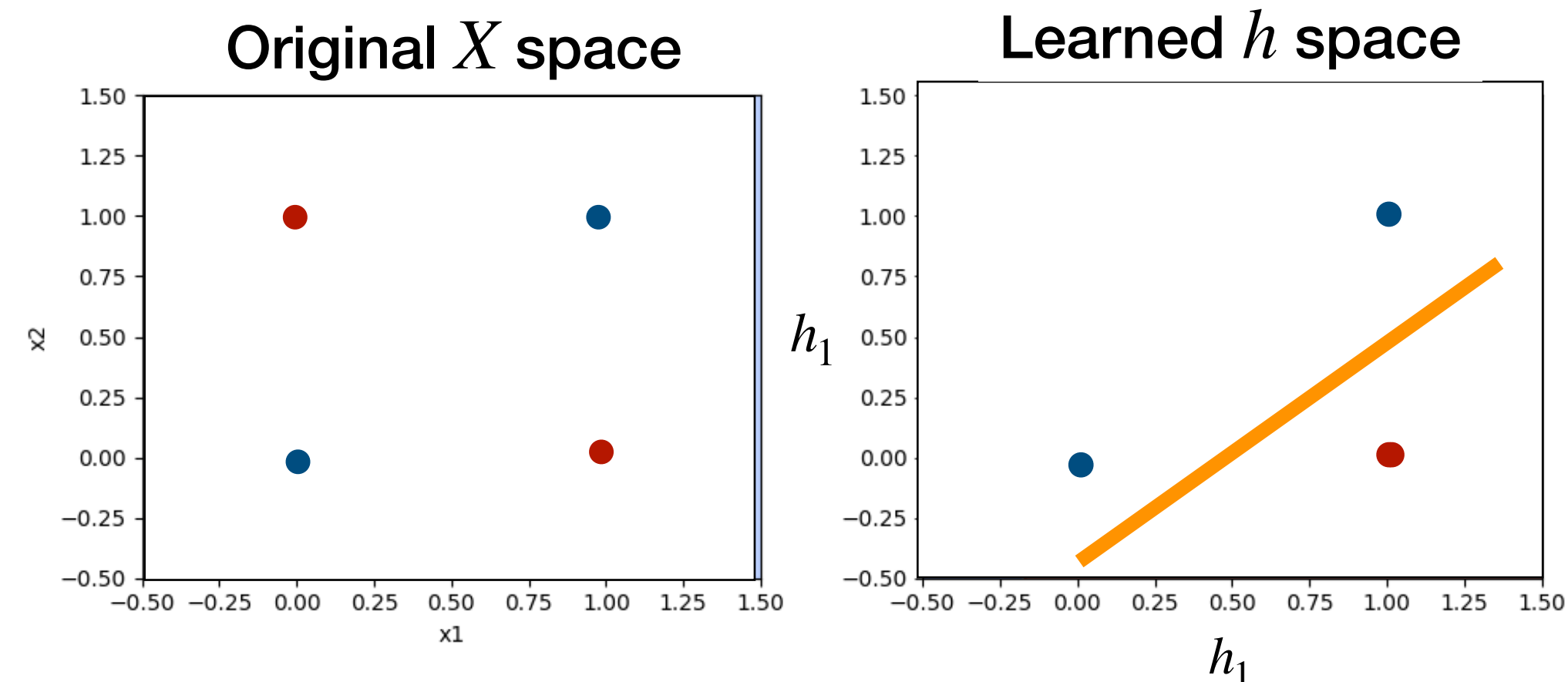
See also Goodfellow Book Section 6.1 for a ReLU-activated MLP construction

Universal Approximation Theorem (UAT)

- Consider an **MLP** with a **single hidden layer**, **sigmoid hidden activation** and **linear output activation**

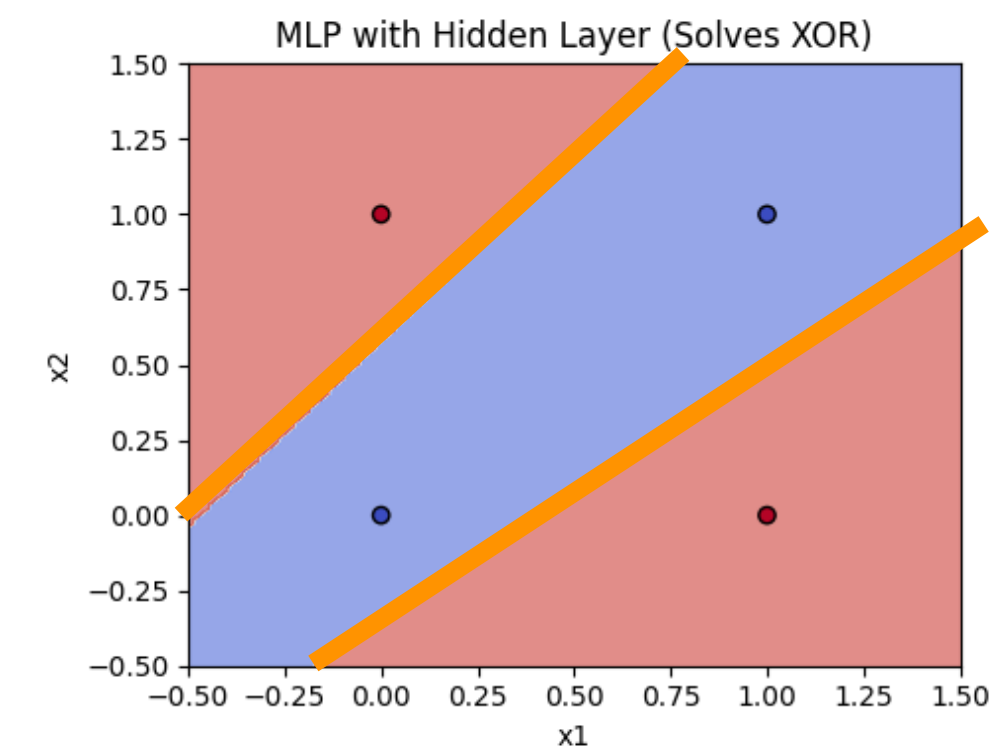
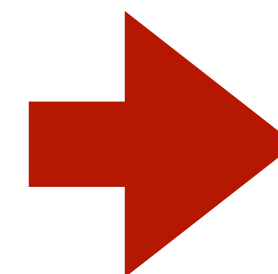
$$h^{(1)} := \sigma(W^{(1)}x) \quad // \text{ we have 2-hidden units - } h \text{ is 2-D so we can easily plot this}$$
$$o := \text{linear}(Wh^{(1)})$$

- $h^{(1)} := \sigma(W^{(1)}x)$ maps the linearly in-separable XOR input to a new linearly separable feature-space in $\mathbb{R}^2$ $\therefore$



This is the input to final layer
(classification head)

$$o := \text{linear}(Wh^{(1)})$$



See also Goodfellow Book Section 6.1 for a ReLU-activated MLP construction

Exponential Representation

Advantage of Depth

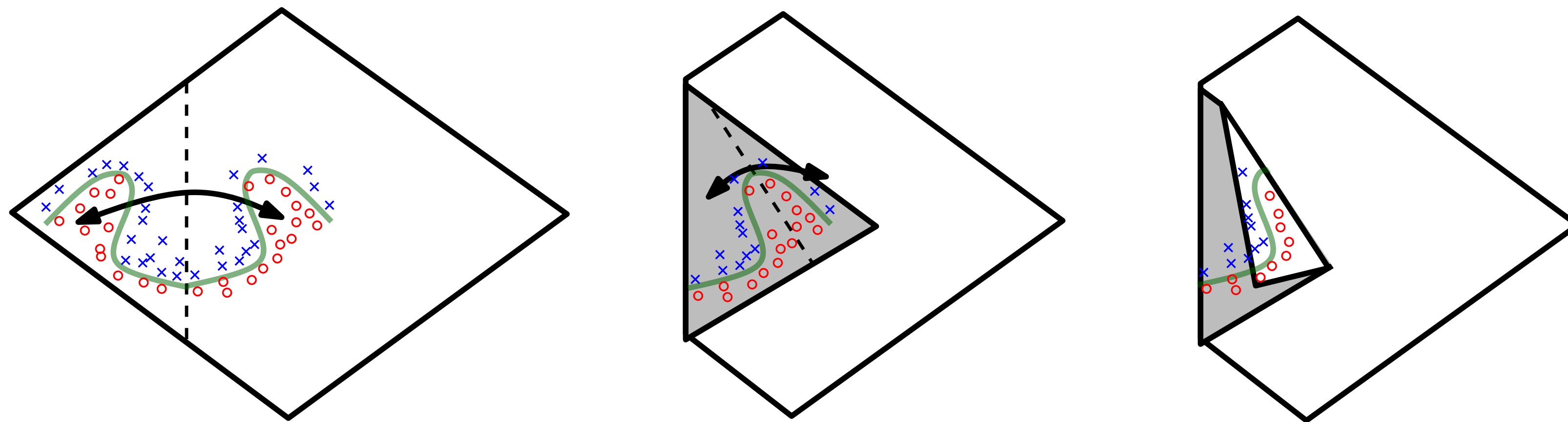


Figure 6.5

Better Generalization with Greater Depth

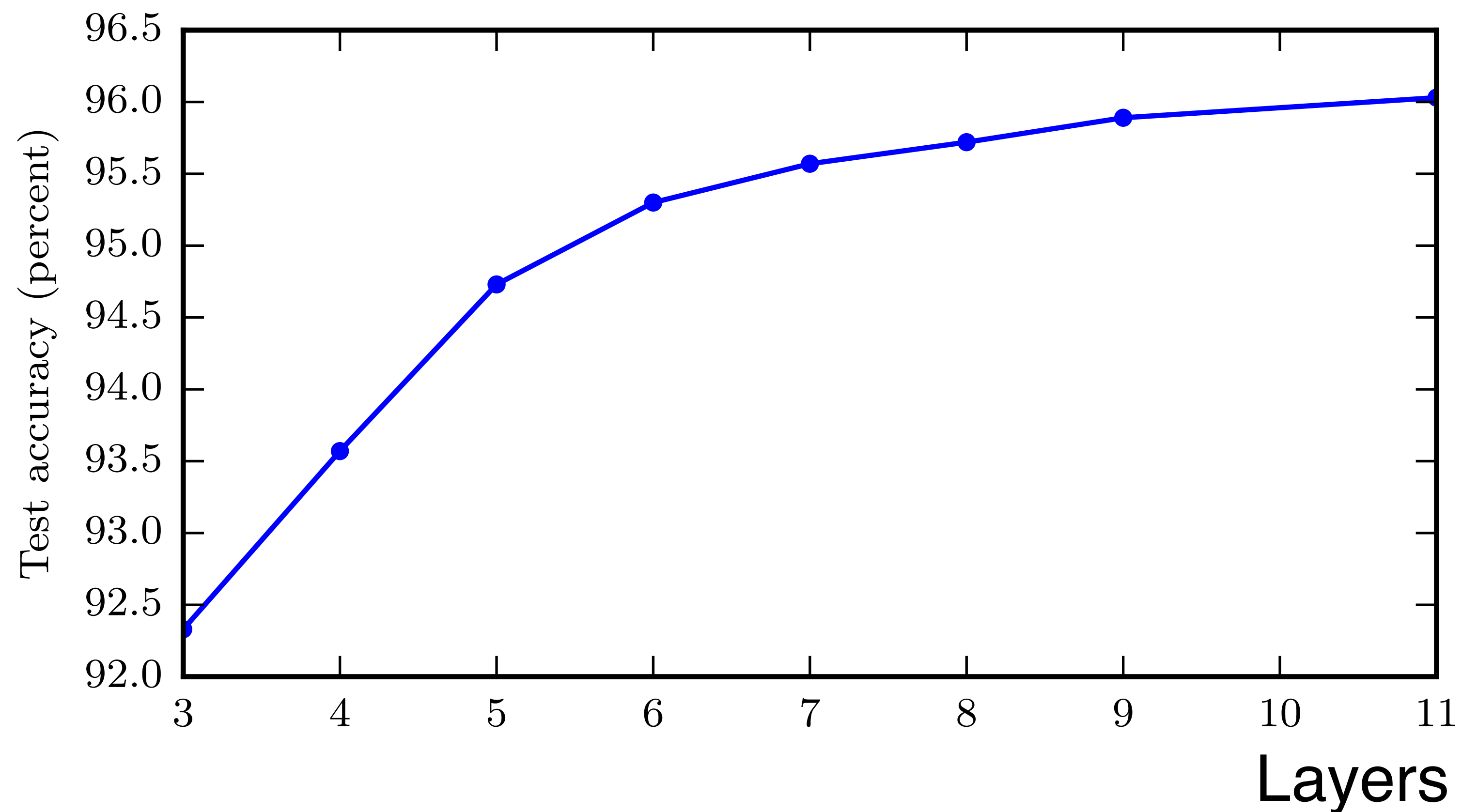


Figure 6.6

Large, Shallow Models Overfit More

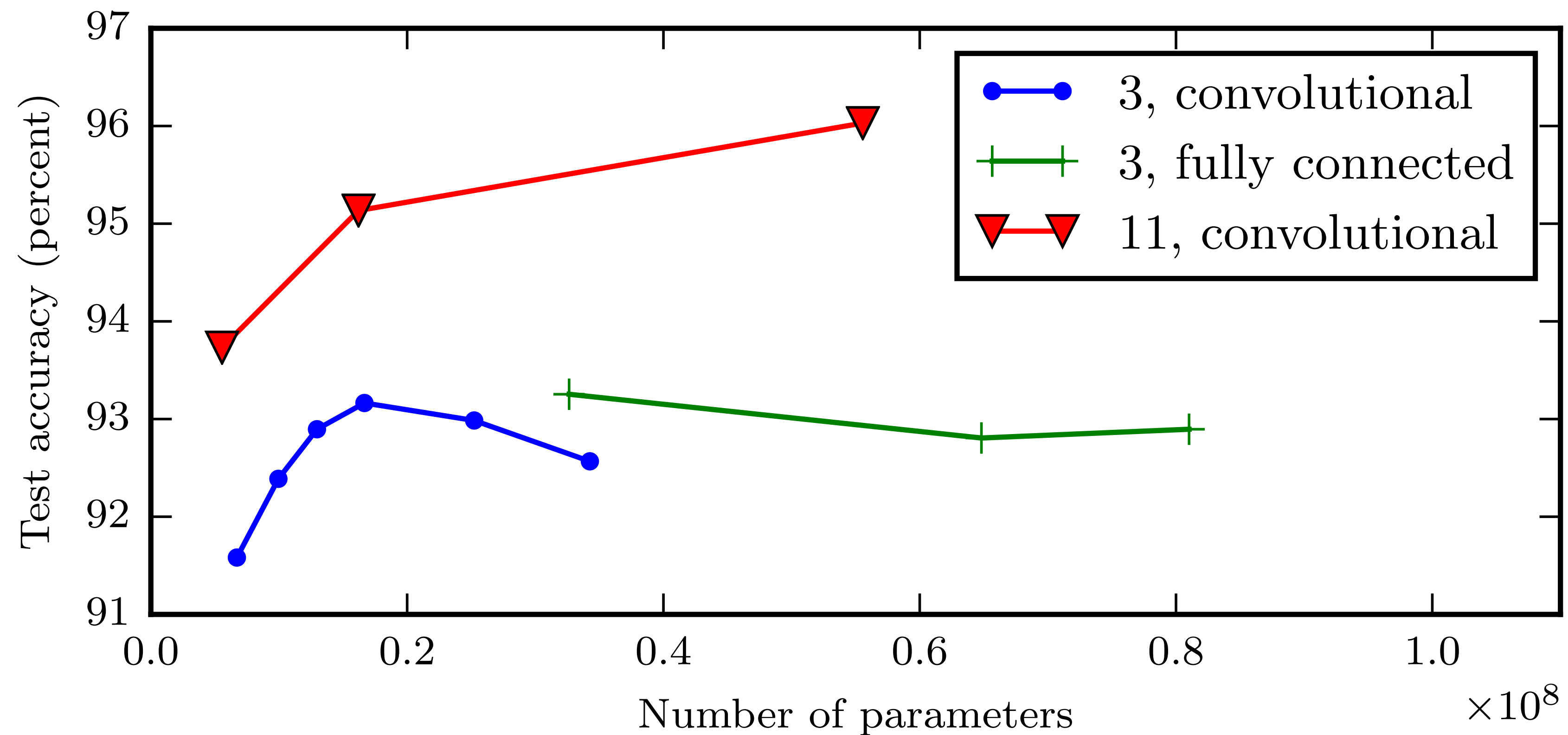
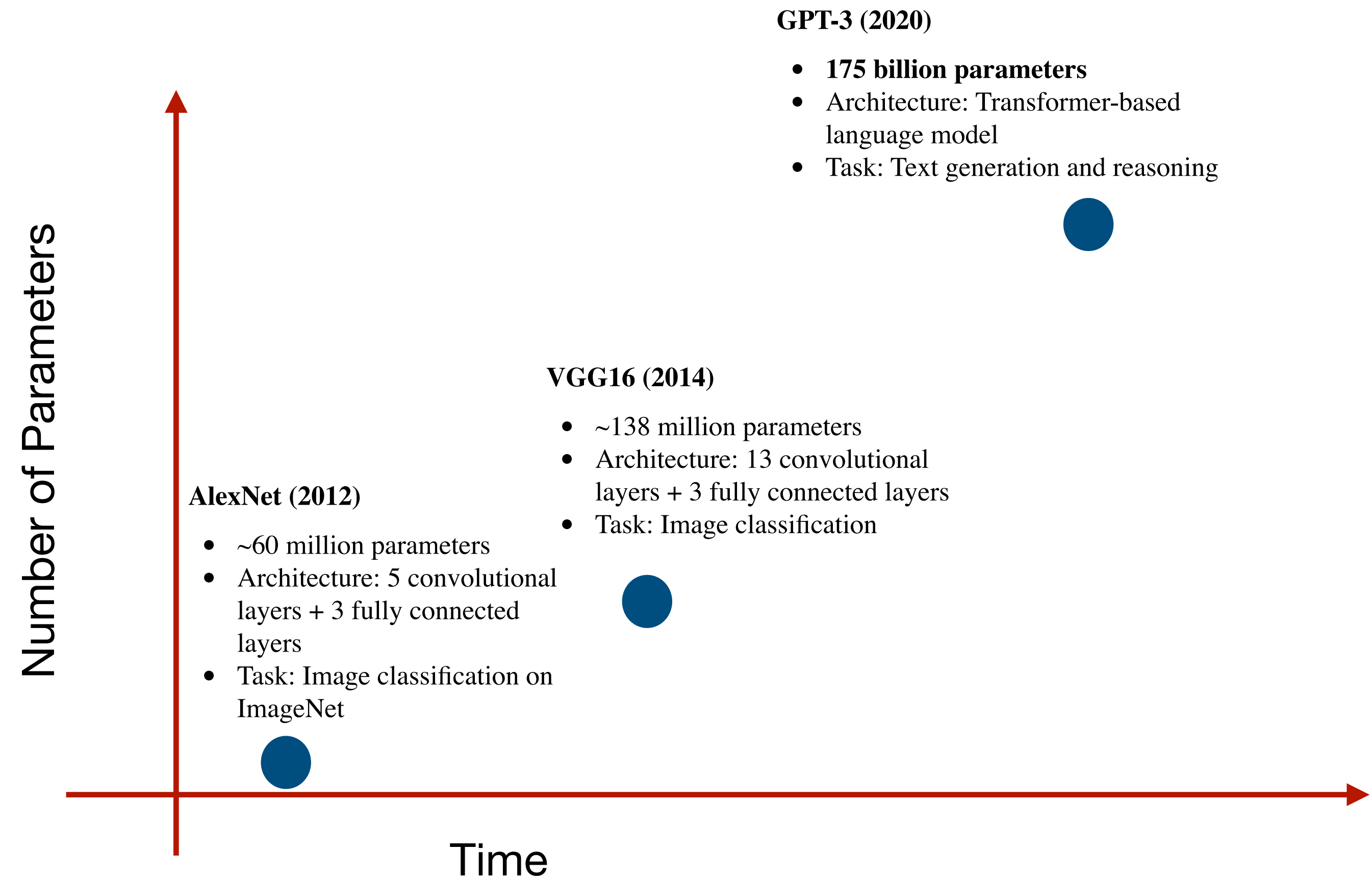


Figure 6.7

Backpropagation

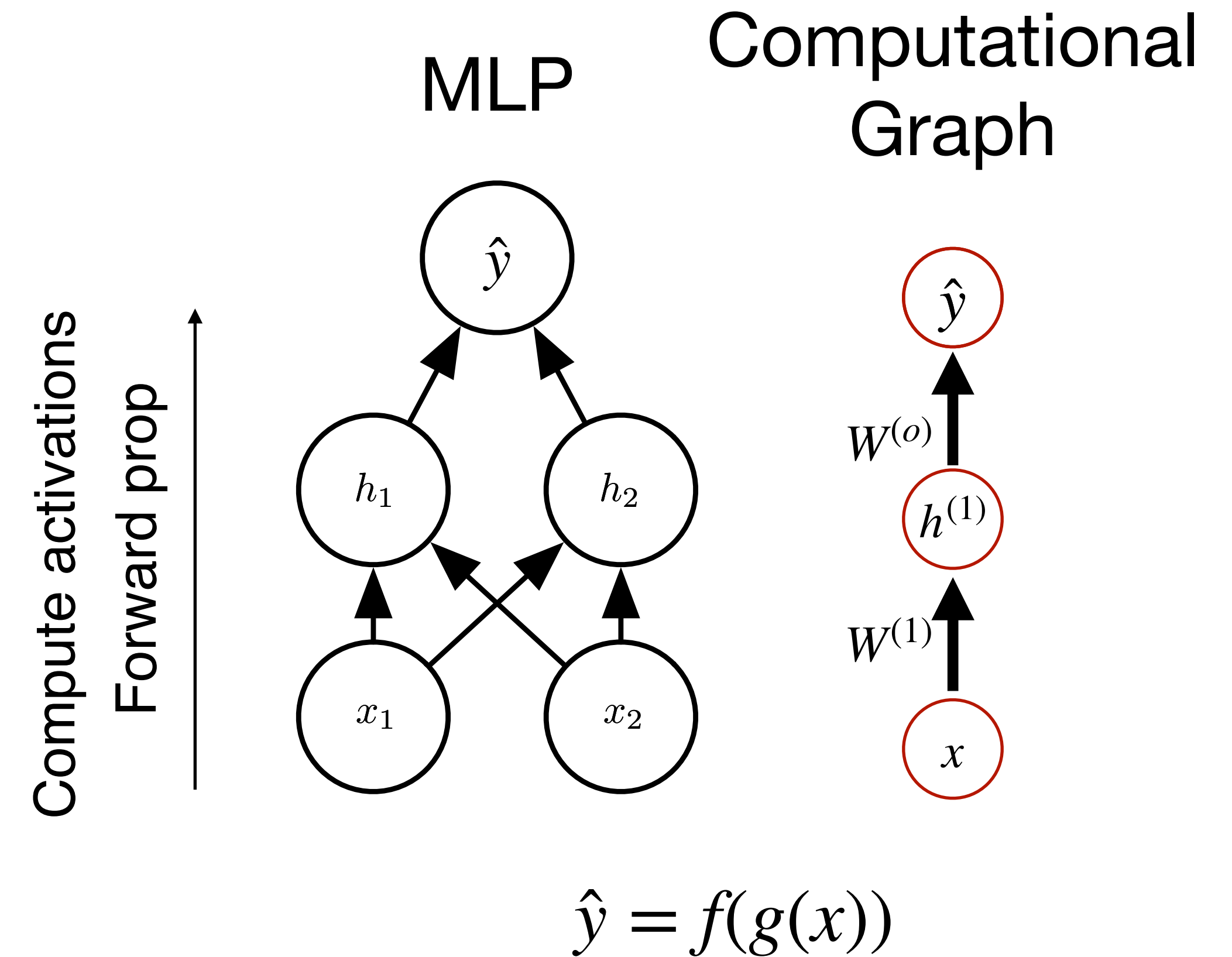
Motivation

- We need an efficient way to optimize parameters of the network
- Back-propagation (Rumelhart *et al.*, 1986a)



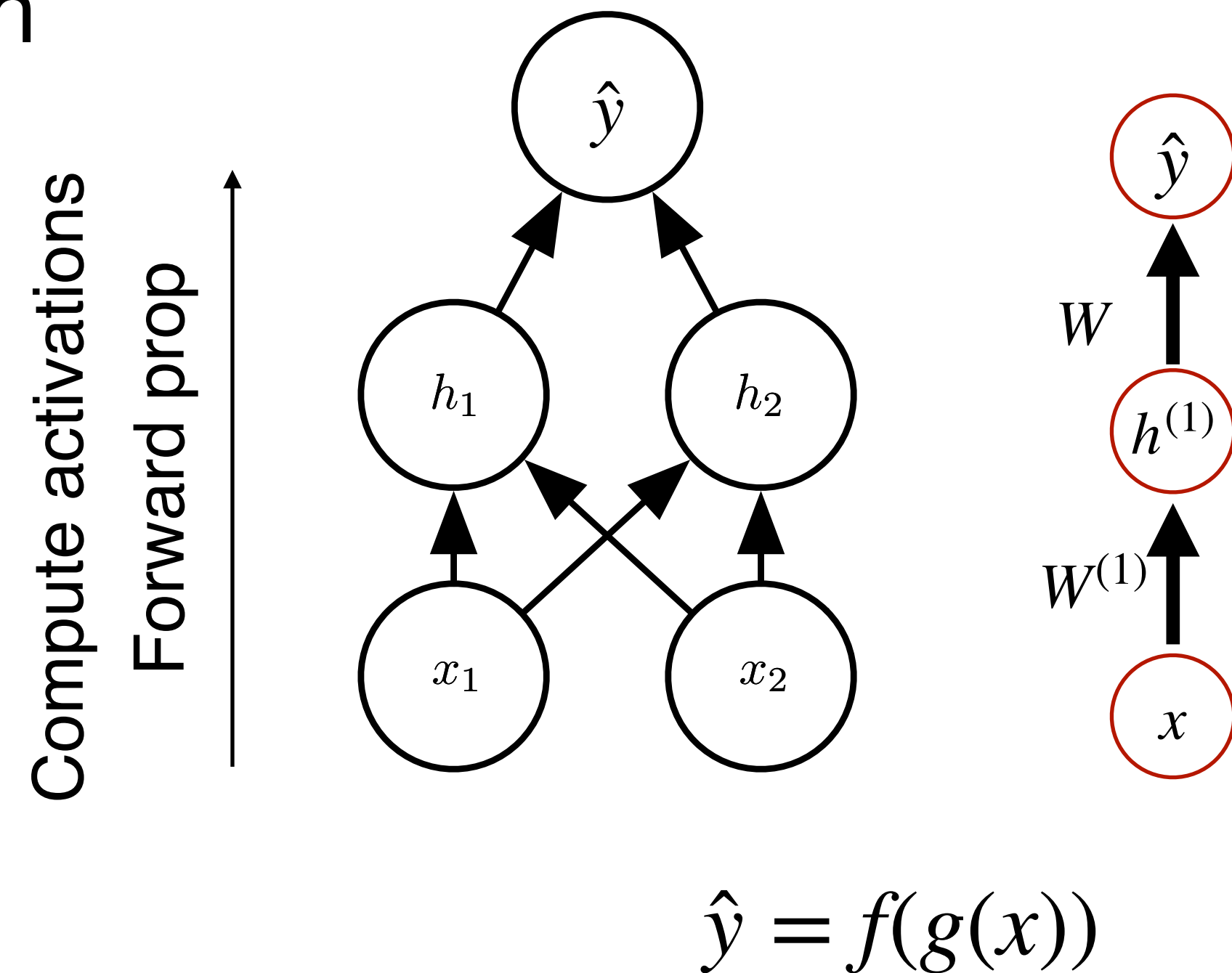
Back-Propagation Overview

- **Forward propagation:** input x passes up to the hidden units of each layer producing $\hat{y}$



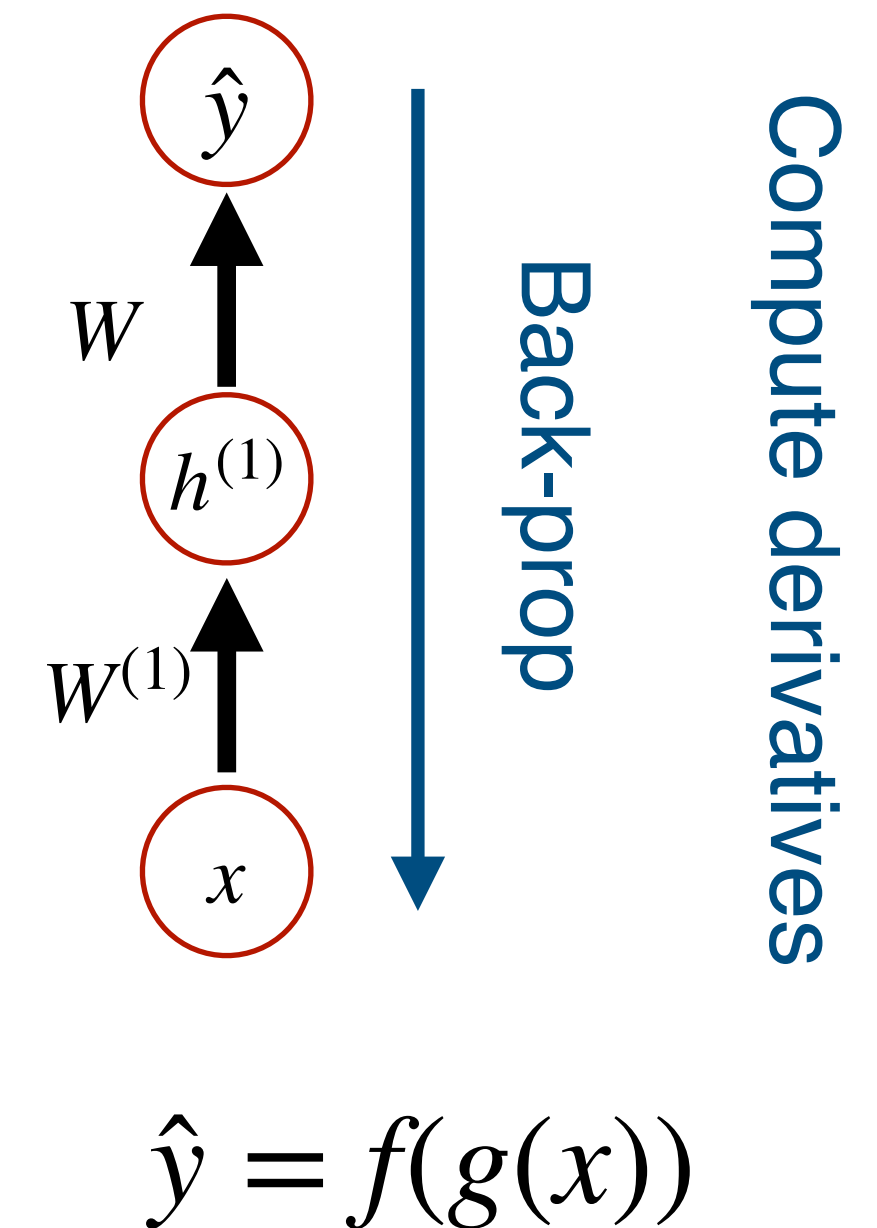
Back-Propagation Overview

- **Forward propagation:** input x passes up to the hidden units of each layer producing $\hat{y}$
- **Compute the scalar cost $J(\theta)$**
 - e.g. $J(\theta) = L(\hat{y}, y) + \lambda\Omega(\theta)$,
where $L(\hat{y}, y) = -\log p(y | x; \theta)$



Back-Propagation Overview

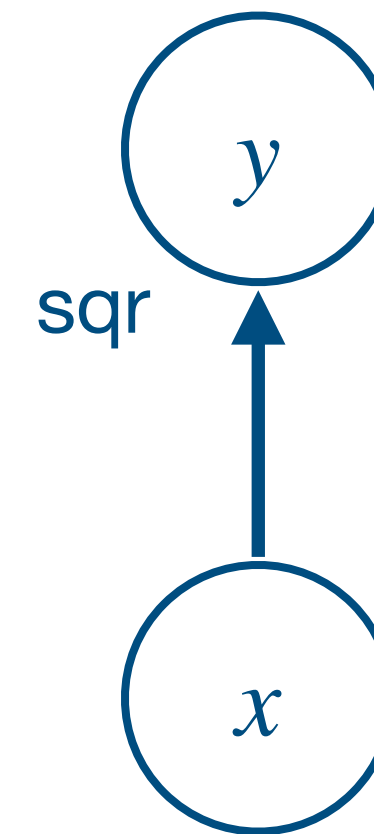
- **Forward propagation:** input x passes up to the hidden units of each layer producing $\hat{y}$
- **Compute the scalar cost $J(\theta)$**
 - e.g. $J(\theta) = L(\hat{y}, y) + \lambda\Omega(\theta)$
- **Backprop:** Compute gradients of loss w.r.t. weights $\nabla_{\theta}J(\theta)$
 - Gradients **flow from cost backwards** through the network
 - Applies a simple and efficient algorithm
- **SDG** as the learning algorithm which **uses the gradients**



Computation Graphs

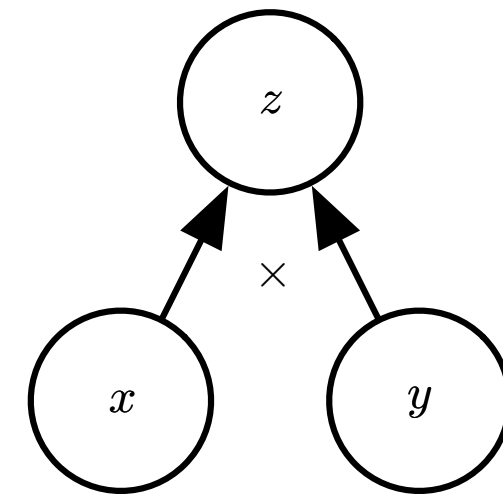
e.g. y is computed from x as
the square of x , $y = x^2$

- **ANNs as computational graphs**
- **Nodes are variables** (scalar, vector, tensor)
- **Operations are simple functions** of one or more variables



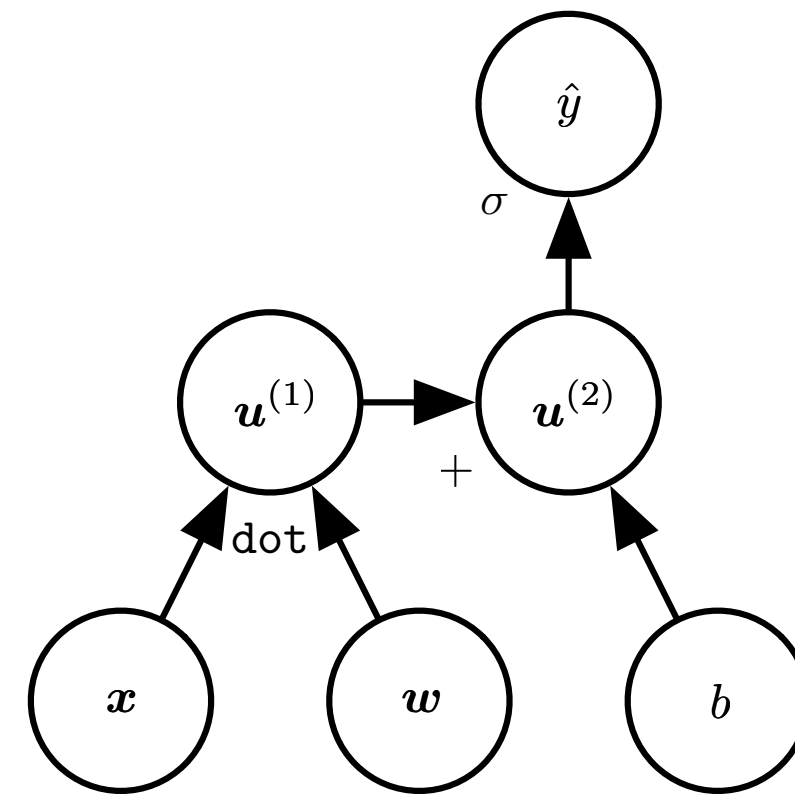
Computation Graphs

Multiplication



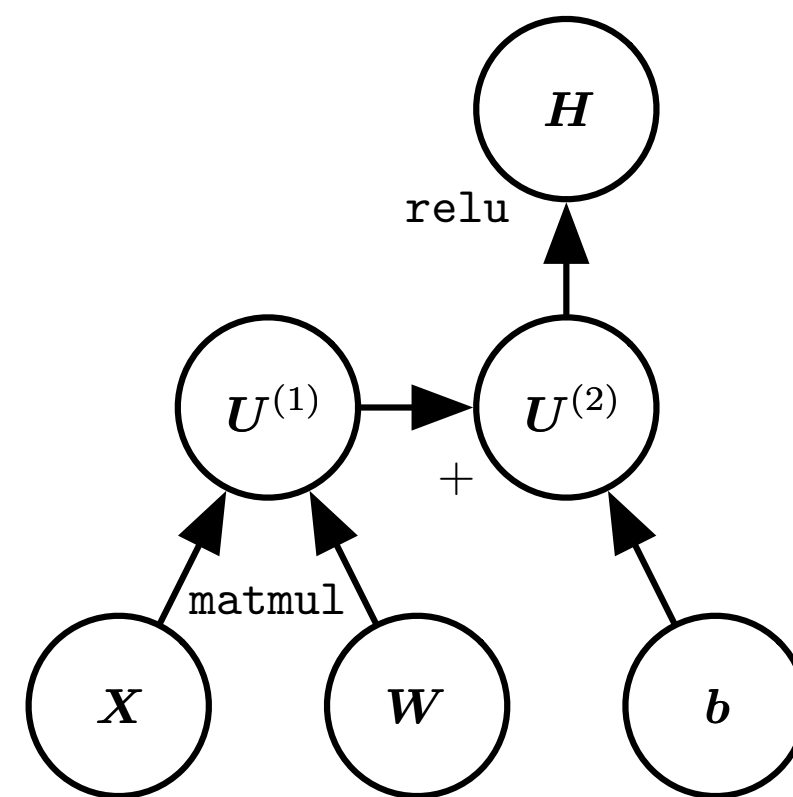
(a)

Logistic regression



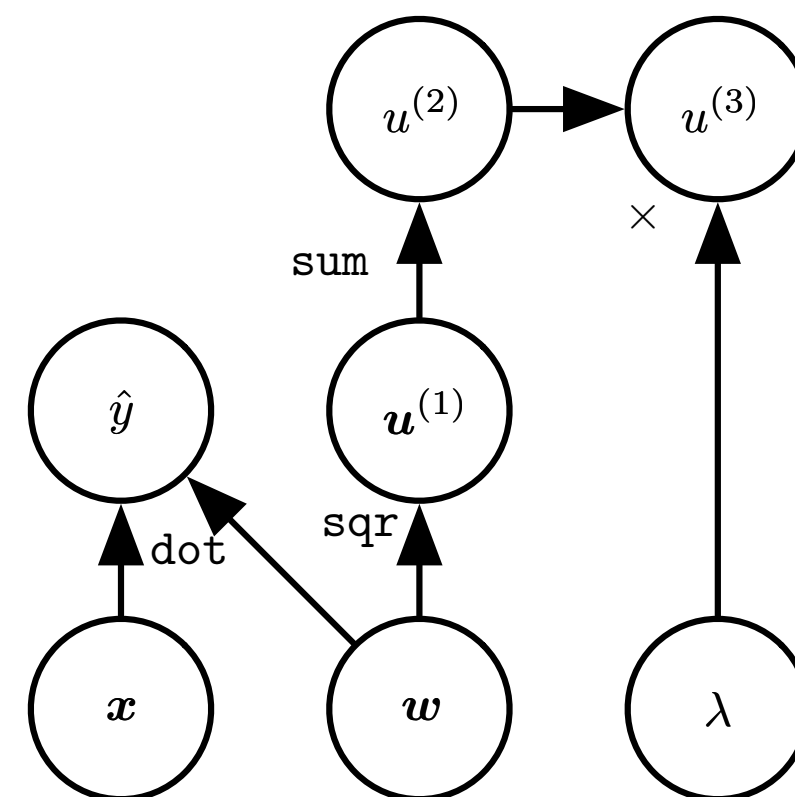
(b)

ReLU layer



(c)

Linear regression
and weight decay



(d)

Figure 6.8

Chain Rule of Calculus

- We need the **chain rule** because ANNs are **functions formed of other function**

- Example: $z = f(g(x))$, where
 $g(x) = \sin(x) + 2$,
 $f(y) = y^2 - 1$

- The chain rule:

- $\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$

- Therefore, $z = g(x)^2 - 1 = (\sin(x) + 2)^2 - 1$

- Compute derivative using chain rule

$$\frac{dz}{dx} = \frac{d}{dx}[g(x)^2 - 1] = 2 \cdot g(x) \cdot \frac{dg(x)}{dx}$$

- Since

$$g(x) = \sin(x) + 2, \quad \frac{dg(x)}{dx} = \cos(x)$$

- Thus

$$\frac{dz}{dx} = 2g(x) \cos(x) = 2(\sin(x) + 2) \cos(x)$$

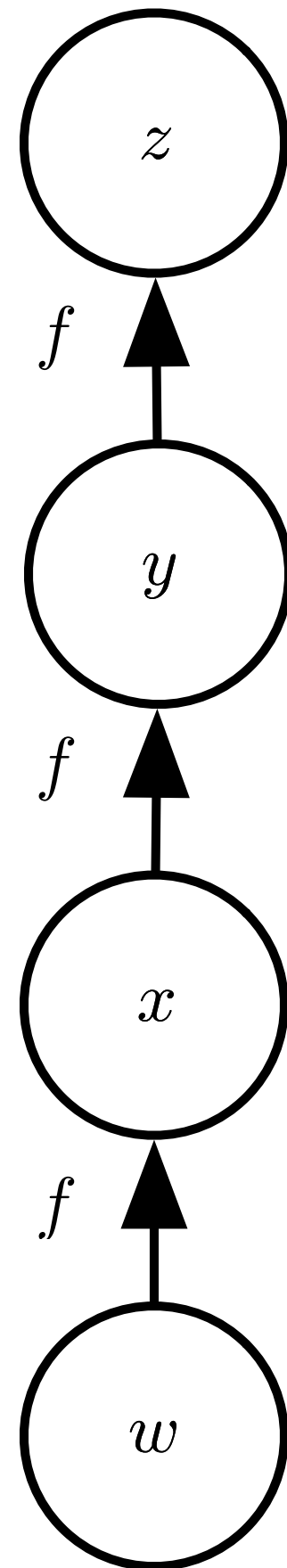
Chain Rule of Calculus

- The chain rule - **generalization to beyond scalars:**

- $\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$, or in vector notation $\nabla_{\mathbf{x}}^z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^\top \nabla_{\mathbf{y}}^z$, where $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ is an $n \times m$ Jacobian matrix of g

- Therefore, **the gradient of a variable $\mathbf{x}$** is obtained by **multiplying** the **Jacobian matrix $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$** by the **gradient $\nabla_{\mathbf{y}}^z$**
- **Backprop** efficiently performs this for each operation in the graph

Repeated Subexpressions



$$\frac{\partial z}{\partial w} \tag{6.50}$$

$$= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w} \tag{6.51}$$

$$= f'(y) f'(x) f'(w) \tag{6.52}$$

$$= f'(f(f(w))) f'(f(w)) f'(w) \tag{6.53}$$

Back-prop avoids computing this twice

Figure 6.9

Symbol-to-Symbol Differentiation

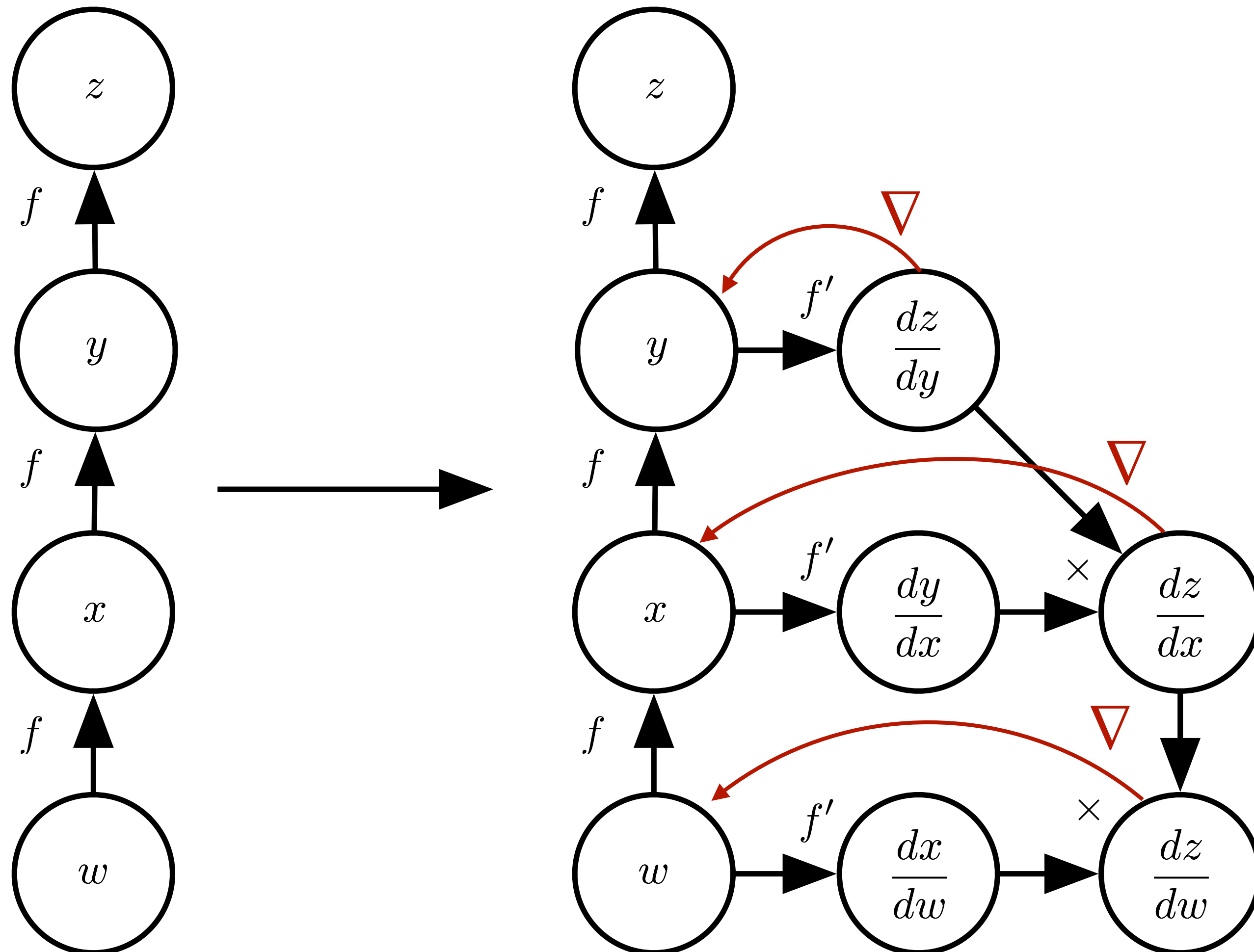


Figure 6.10

- Backprop **adds nodes** to the computational graph **describing how to calculate the derivative**
- **Generic graph evaluation engine will calculate them on demand** based on the current loss to produce the gradient ∇ for each variable

Neural Network Loss Function

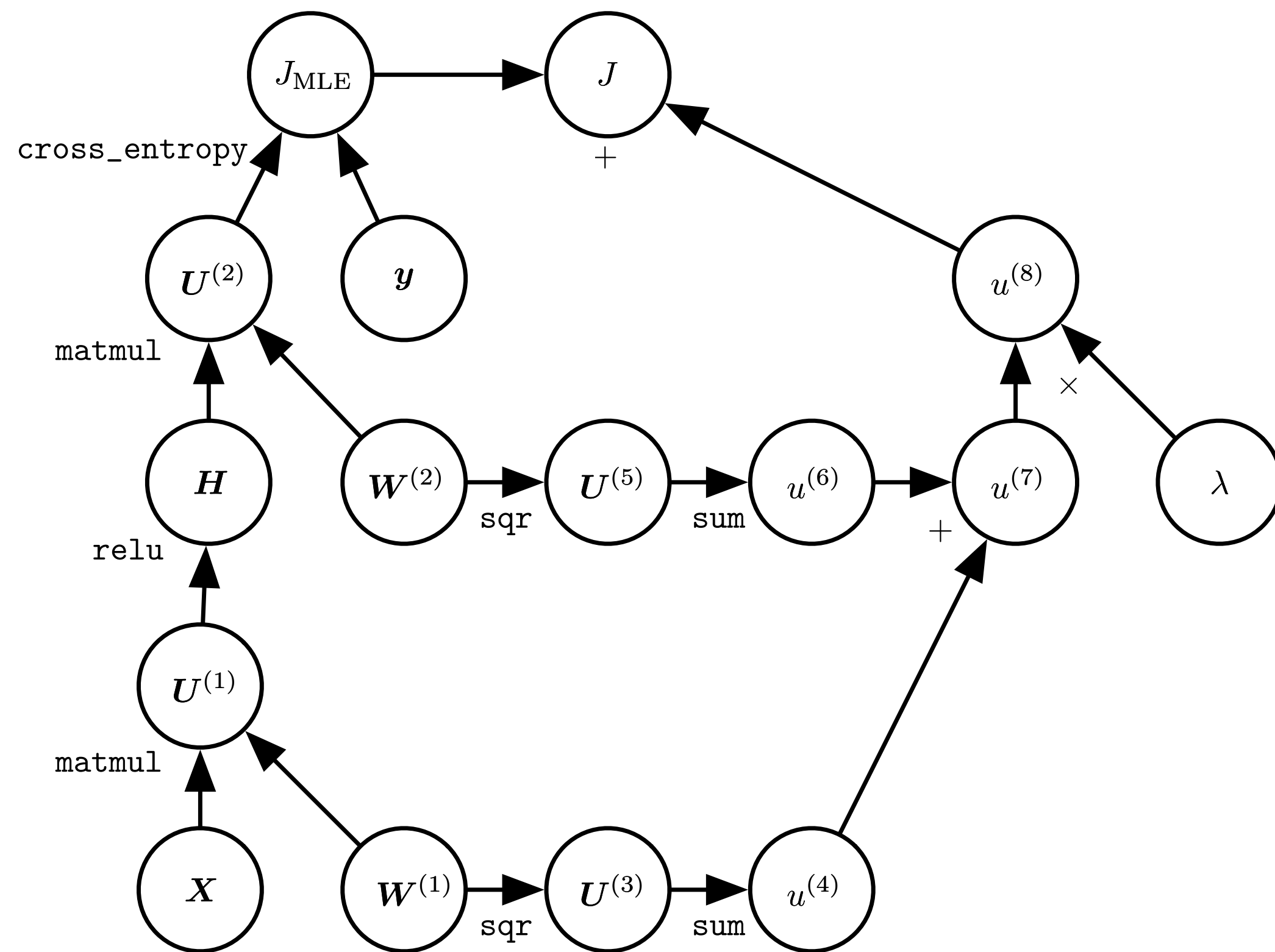


Figure 6.11

- Computational graph for **two layer MLP with with cost function**
- Minimize **cross-entropy loss** as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Neural Network Loss Function

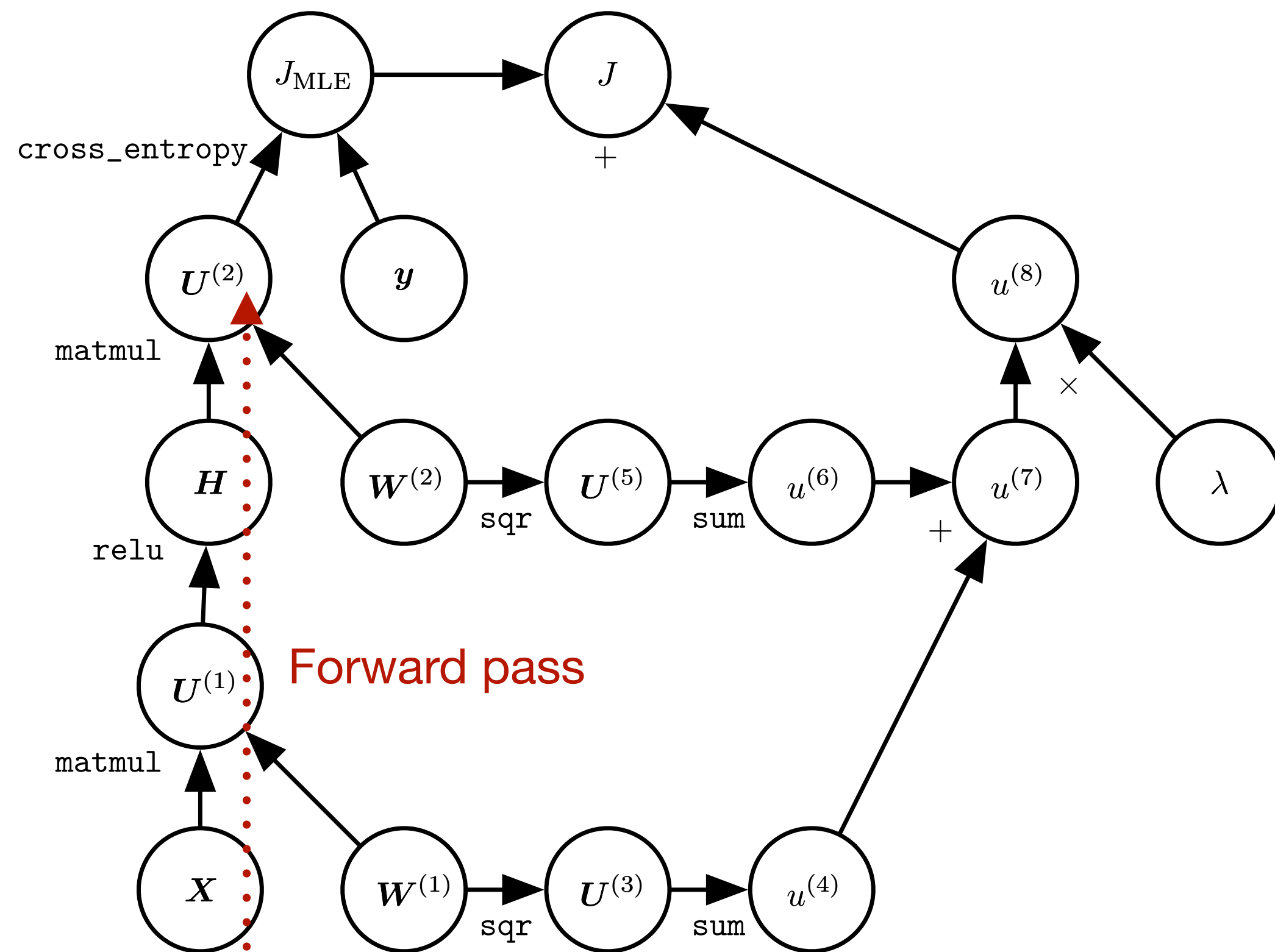


Figure 6.11

- Computational graph for two layer MLP with with cost function
- Minimize cross-entropy loss as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Neural Network Loss Function

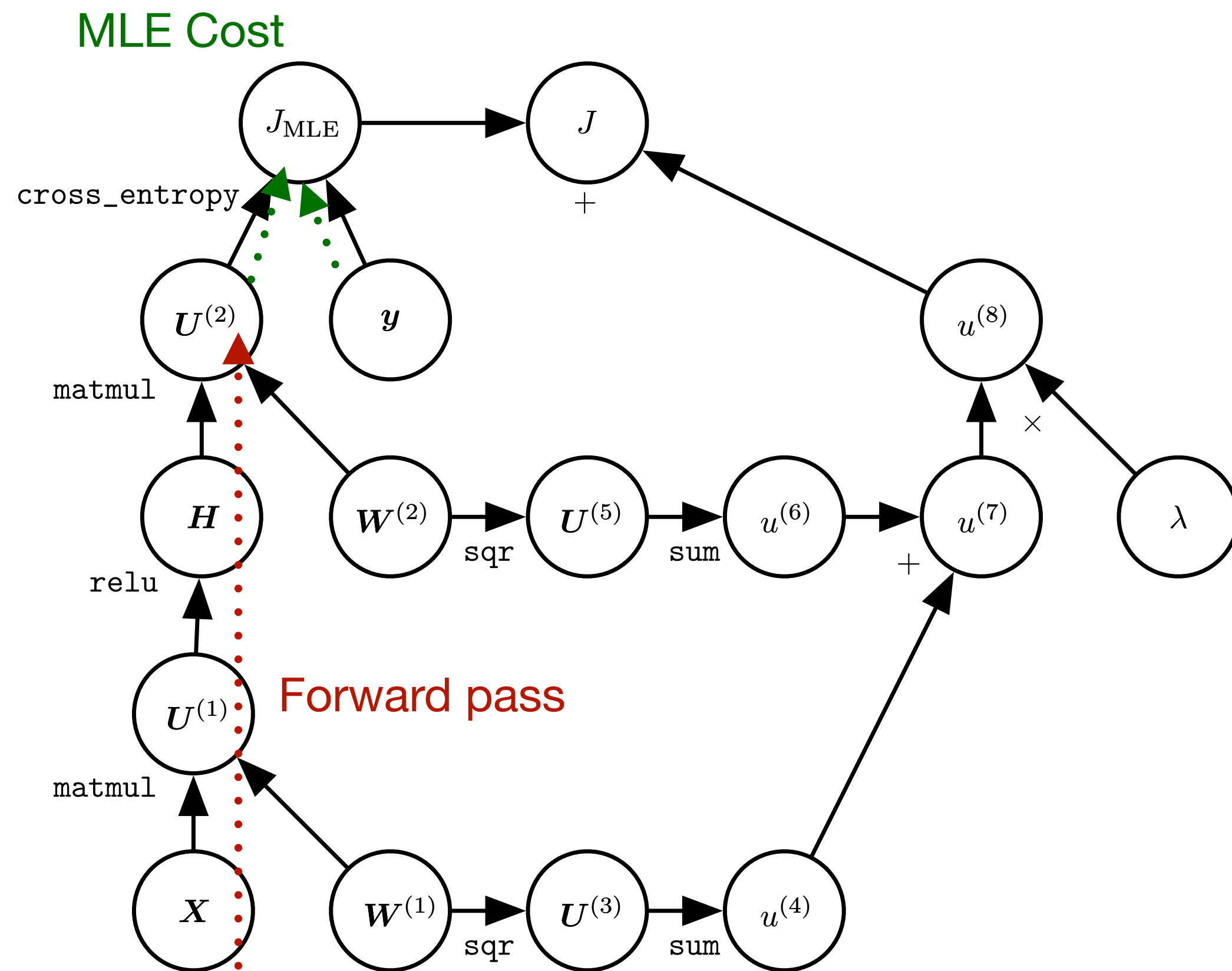


Figure 6.11

- Computational graph for two layer MLP with with cost function
- Minimize cross-entropy loss as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Neural Network Loss Function

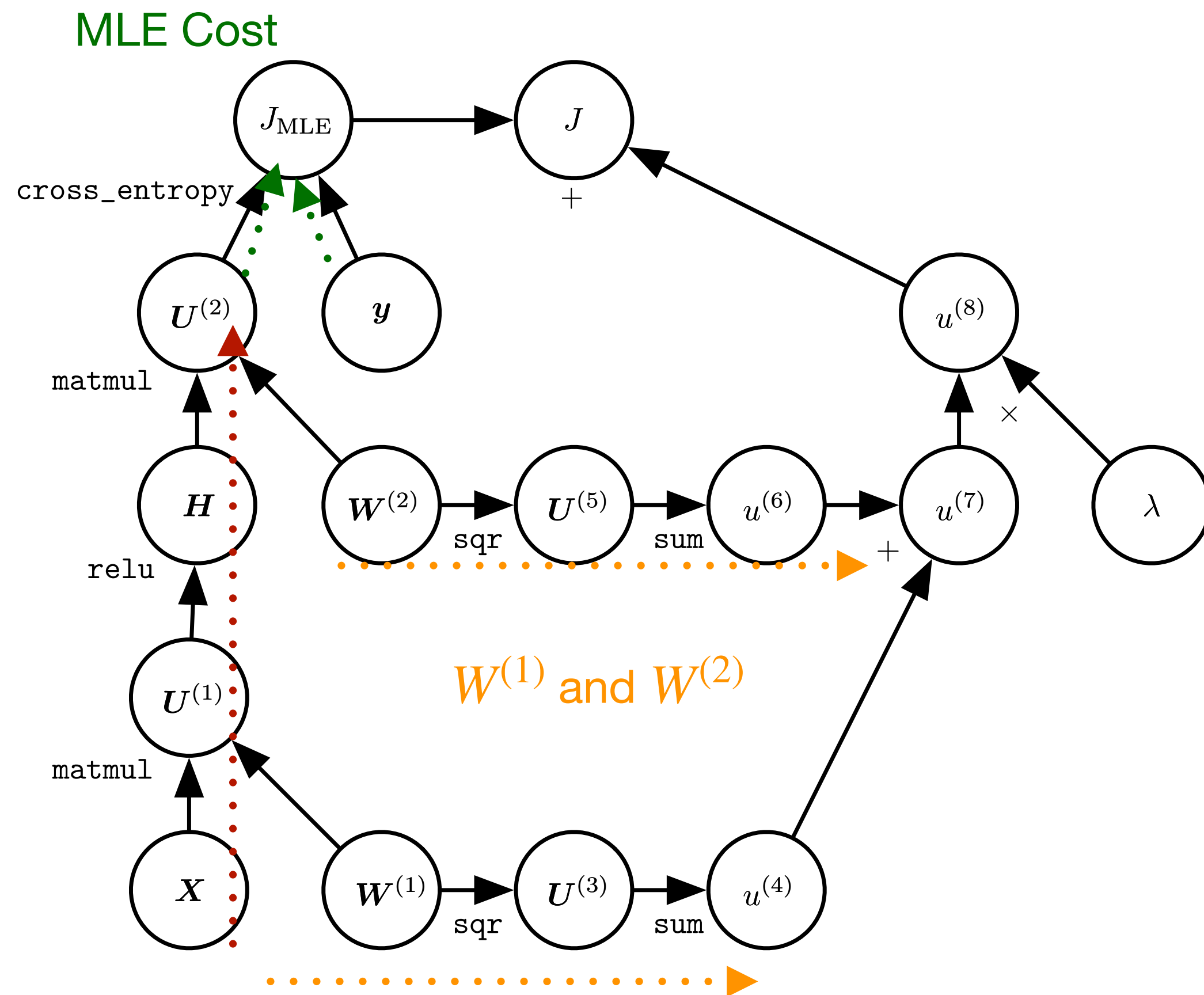


Figure 6.11

- Computational graph for two layer MLP with with cost function
- Minimize cross-entropy loss as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Neural Network Loss Function

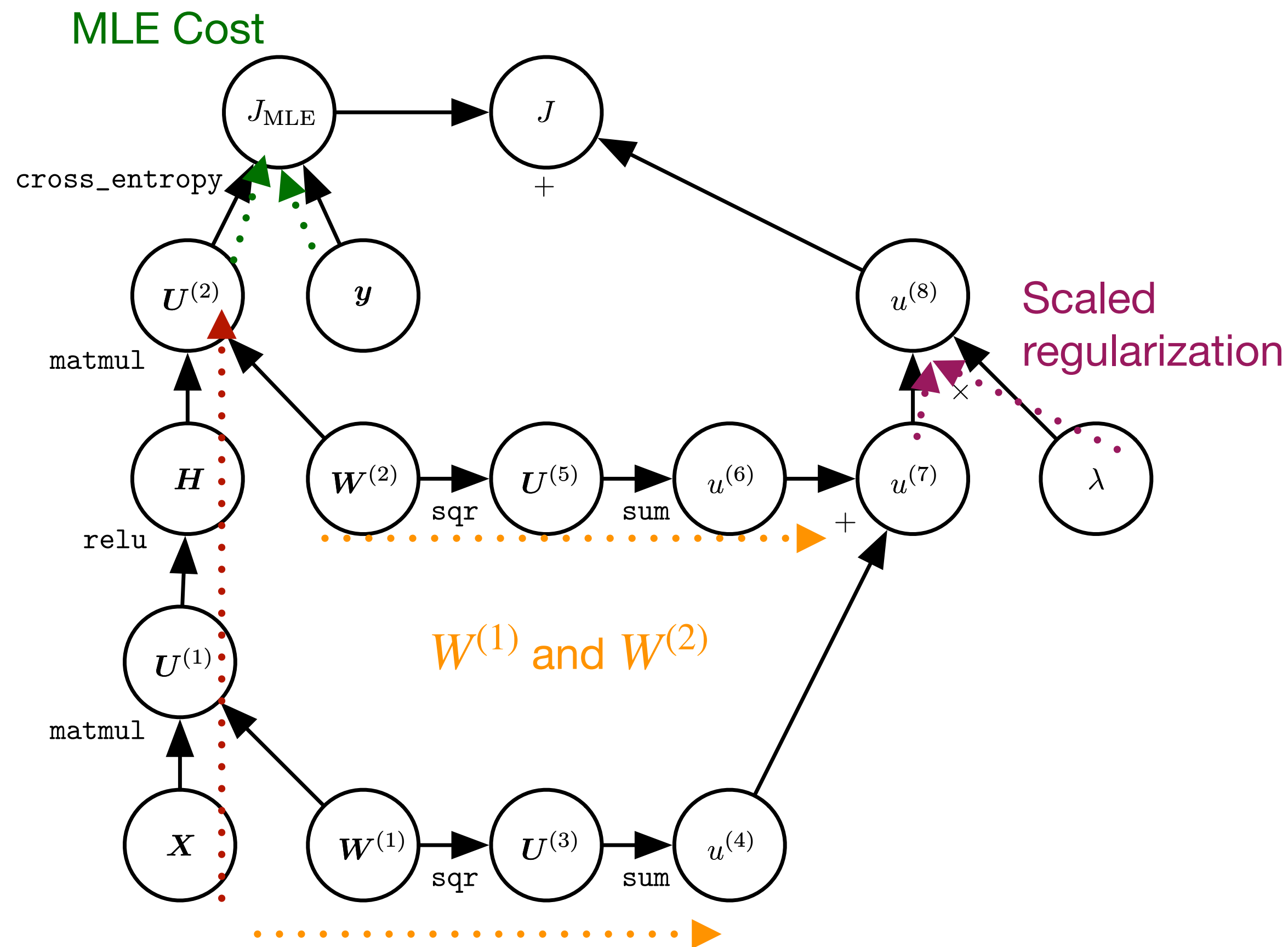


Figure 6.11

- Computational graph for two layer MLP with with cost function
- Minimize cross-entropy loss as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Neural Network Loss Function

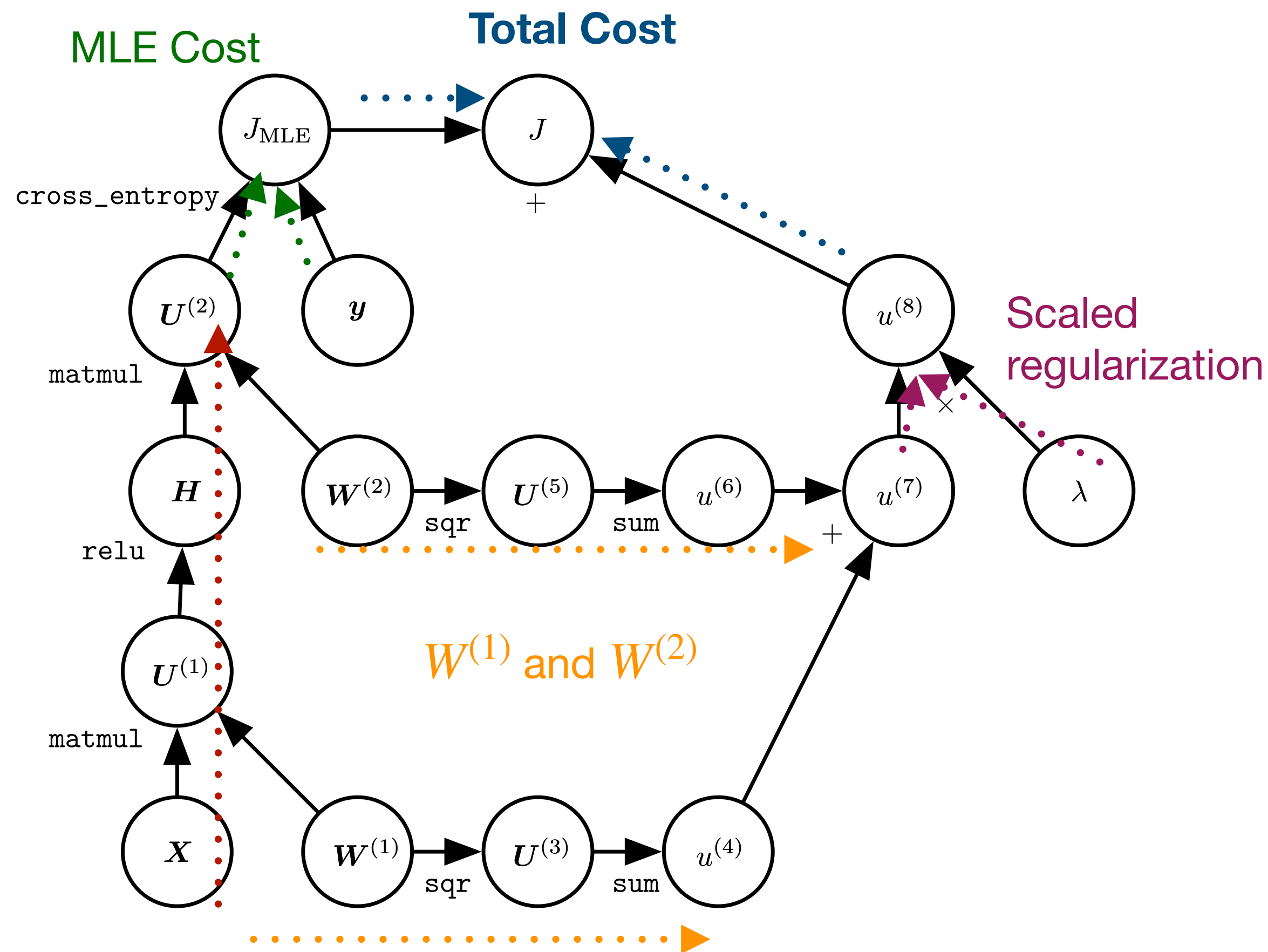


Figure 6.11

- Computational graph for two layer MLP with with cost function
- Minimize cross-entropy loss as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Neural Network Loss Function

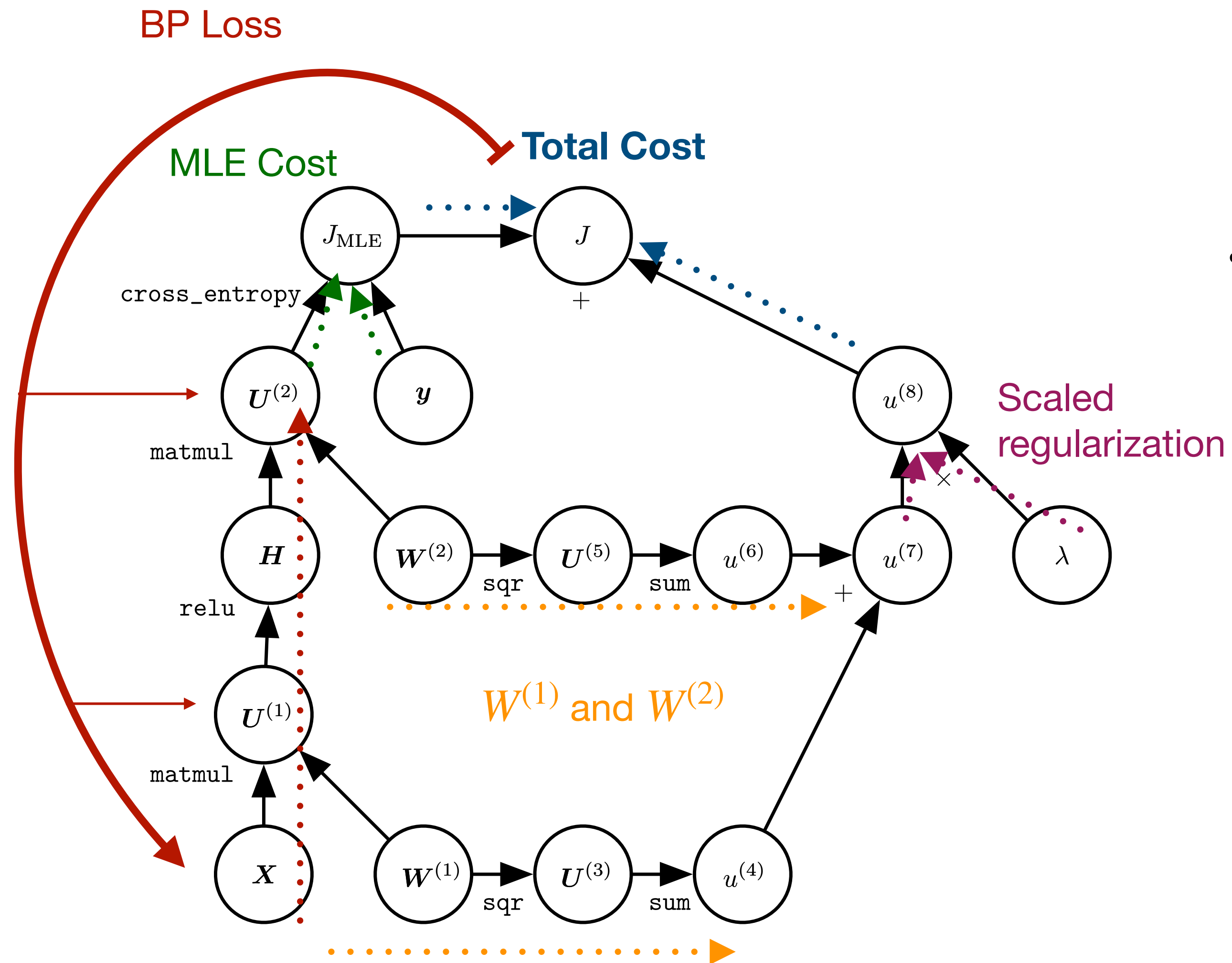


Figure 6.11

- Computational graph for two layer MLP with with cost function
- Minimize cross-entropy loss as maximum likelihood estimate (MLE)

$$J = J_{MLE} + \lambda \left(\sum_{i,j} (W_{i,j}^{(1)})^2 + \sum_{i,j} (W_{i,j}^{(2)})^2 \right)$$

Gradient Norm

- **L2 norm of the gradient vector**

- Provides important **insights into the optimization process** in deep learning:

- **Training Progress**

- Large gradient norm $\rightarrow$ parameters are far from a minimum; updates will be large.
 - Small gradient norm $\rightarrow$ near a stationary point (could be a minimum, maximum, or saddle point).

- **Learning Rate Issues**

- If gradient norm is huge $\rightarrow$ risk of exploding gradients; may need smaller learning rate or gradient clipping.
 - If gradient norm is near zero early in training $\rightarrow$ possible vanishing gradients; may need better initialization or activation functions.

- **Detecting Saddle Points**

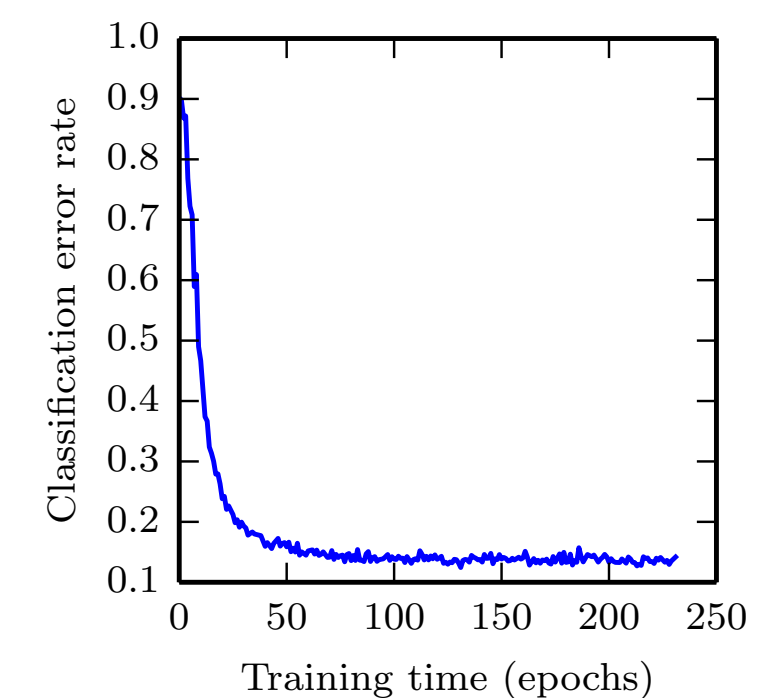
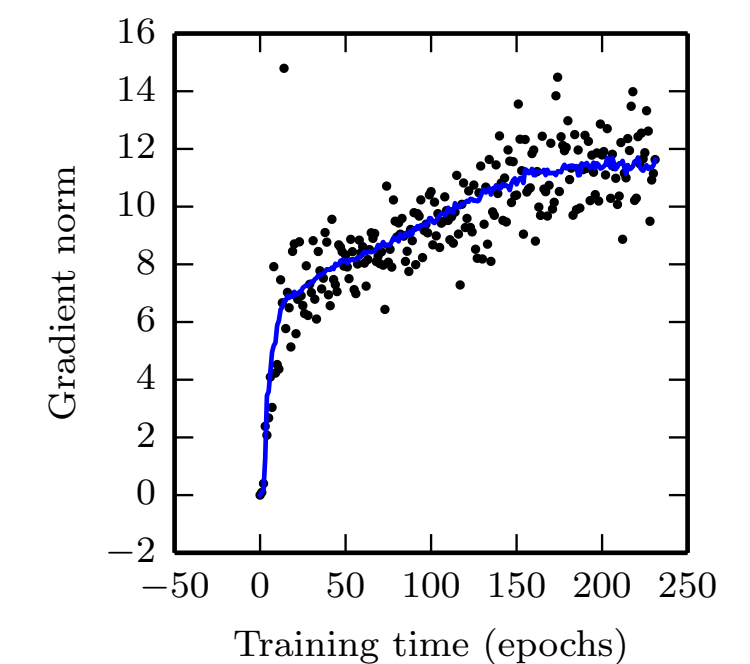
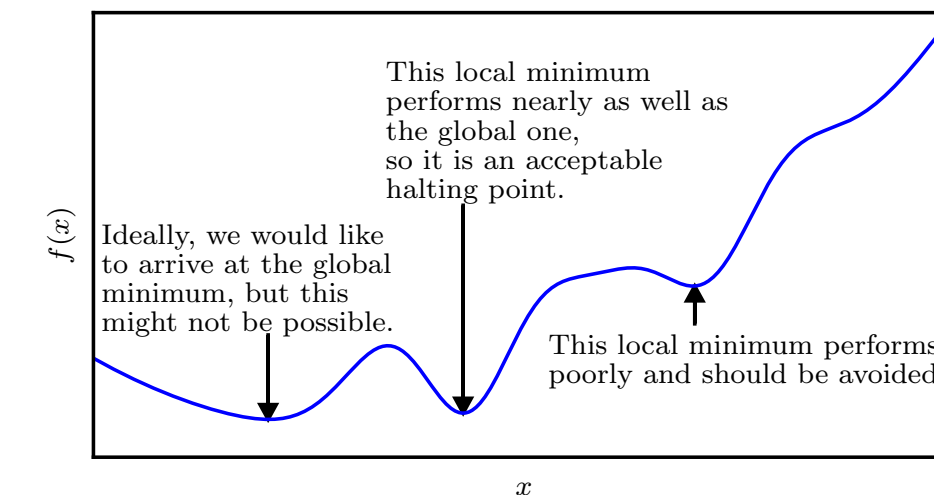
- Gradient norm close to zero but Hessian has mixed signs $\rightarrow$ likely a saddle point, not a true minimum.

- **Convergence Monitoring**

- Gradient norm approaching zero is often used as a stopping criterion for optimization.

- **Diagnosing Pathologies**

- Persistent large norms $\rightarrow$ optimization instability.
 - Persistent tiny norms $\rightarrow$ stuck in flat regions or poor architecture choices.



To Do

- What I plan to do:
 - Convolutional Neural Networks (CNNs)
- What I recommend YOU to do for next class:
 - CNNs: Goodfellow chapters 9, Bishop chapters 10
 - Readings
 - Latent Diffusion Planning for Imitation Learning: <https://arxiv.org/pdf/2504.16925>, or
 - Loss of plasticity in deep continual learning: <https://www.nature.com/articles/s41586-024-07711-7>
 - Deep Reinforcement Learning and the Deadly Triad: <https://arxiv.org/pdf/1812.02648>
 - Breaking the Deadly Triad with a Target Network: <https://proceedings.mlr.press/v139/zhang21y/zhang21y.pdf>
- **Homework 1: JAX Versus Pytorch MLP - Due February 13, 2026**